

Algorithms and Data Structures

Complete Self-Study Lecture Notes

Topics 1-80

Contents

1	The RAM model and asymptotic running time	10
1.1	Motivation	10
1.2	Running time	10
2	Asymptotic notation: O, Ω, Θ	11
2.1	Definitions	11
2.2	Independence of the starting index	11
3	Range-sum queries in a static array: prefix sums	12
3.1	Problem	12
3.2	Complexity	12
4	Membership in a sorted array: binary search	13
4.1	Problem	13
4.2	Core idea	13
5	The sorting problem: two formulations and equivalence	14
5.1	Two formulations	14
5.2	Equivalence	14
6	The formula $\log(n!) = \Theta(n \log n)$	14
7	Lower bound for comparison sorting	15
7.1	Comparison model	15
7.2	Decision-tree argument	15
8	Merge Sort	16
8.1	Core idea	16
9	Solving $T(n) = 2T(n/2) + O(n)$	17
9.1	Recursion-tree proof	17
10	Counting inversions in an array	18
10.1	Naive algorithm	18
10.2	Divide-and-conquer algorithm	18

11	Quick Sort	18
11.1	Core idea	18
11.2	Partition invariant	19
12	Randomized Quick Select	19
12.1	Problem	19
12.2	Core idea	20
13	Deterministic linear-time selection	20
13.1	Median of medians	20
13.2	Why the pivot is good	21
14	Deterministic Quick Sort in $O(n \log n)$	21
14.1	Core idea	21
15	Stable Counting Sort and sorting pairs	22
15.1	Problem	22
15.2	Correctness and stability	22
15.3	Sorting pairs	23
16	LSD Radix Sort	23
16.1	Core idea	23
17	Singly and doubly linked lists	24
17.1	Singly linked list	24
17.2	Doubly linked list	24
18	Stack data structure: list implementation	25
18.1	Implementation by a singly linked list	25
19	Checking bracket sequences with several bracket types	26
19.1	Problem	26
19.2	Algorithm	26
20	Nearest smaller/greater elements in a static array	26
20.1	Problem	26
20.2	Nearest smaller to the left	27
20.3	Other variants	27
21	Evaluating expressions in Reverse Polish Notation	28
21.1	Algorithm	28
Part I summary table		28

Part I practice problems	29
22 Supporting the minimum in a stack	29
22.1 Problem	29
22.2 Augmented stack	30
23 Queue data structure: linked-list implementation	31
23.1 Definition	31
23.2 Singly linked-list implementation	31
24 Queue on two stacks and minimum in a queue	32
24.1 Queue on two stacks	32
24.2 Amortized complexity	33
24.3 Minimum in a queue	33
25 Binary heap: definition and array representation	33
25.1 Complete binary tree	33
26 siftUp and siftDown in a binary heap	34
26.1 siftUp	34
26.2 siftDown	34
27 Heap operations via siftUp and siftDown	35
27.1 Operations	35
28 Building a heap in linear time: heapify	36
28.1 Bottom-up heap construction	36
29 Heap Sort and the impossibility of constant-time heap operations	37
29.1 Heap Sort using $\mathcal{O}(1)$ extra memory	37
29.2 Why comparison heaps cannot have both operations in $\mathcal{O}(1)$	38
30 Deleting from a heap by pointer	38
30.1 Handles and indices	38
31 Deleting from a heap by value	39
31.1 Without an index map	39
31.2 With an auxiliary map	39
32 Binomial trees and binomial heaps	39
32.1 Binomial trees	39
32.2 Binomial heaps	40
32.3 Advantage over a binary heap	40

33	Operations in a binomial heap	40
33.1	Linking two trees	40
33.2	Merge	41
33.3	Other operations	41
34	Amortized analysis and accounting time	42
34.1	Definition	42
34.2	Aggregate method	42
34.3	Potential method	42
35	The coin method: accounting analysis	43
35.1	Coin method	43
35.2	Example: queue on two stacks	43
36	Vector data structure: dynamic array	43
36.1	Array implementation	43
36.2	Operation costs	44
36.3	Shrinking	44
37	Sparse Table	44
37.1	Model problem: static range minimum query	44
37.2	Construction	45
37.3	Query	45
38	Segment tree: model problem and query time	45
38.1	Model problem	45
38.2	Build and point update	46
38.3	Range query	46
39	Segment tree binary descent: finding a segment with sum at least x	47
39.1	Prefix lower-bound query	47
39.2	Range version	47
40	Segment tree with lazy propagation: range assignment and push	48
40.1	Problem	48
40.2	Lazy assignment tag	48
40.3	Range assignment	48
41	Counting values at most x on a range: Fractional Cascading	49
41.1	Model problem	49
41.2	Merge-sort tree	49
41.3	Fractional Cascading pointers	50

41.4	Query idea	50
Compact complexity table		51
42	Dynamic segment tree	51
42.1	Problem and motivation	51
42.2	Node representation	52
42.3	Point update and range query	52
43	Coordinate compression for segment trees	53
43.1	The idea	53
43.2	Point-based compression	54
43.3	Segment compression for interval lengths	54
43.4	Complexity	54
44	Persistent segment tree	55
44.1	Persistence	55
44.2	Persistent point update	55
45	Counting numbers not exceeding x on a subarray: persistent segment tree solution	56
45.1	Model problem	56
45.2	Prefix versions over values	56
46	Range k-th order statistic via persistent segment trees and parallel descent	57
46.1	Problem	57
46.2	Parallel descent	57
47	Fenwick tree: classical problem, update and getSum	58
47.1	Model problem	58
47.2	The lowbit decomposition	58
47.3	Operations	59
48	Higher-dimensional Fenwick trees	60
48.1	Two-dimensional model problem	60
48.2	Two-dimensional operations	60
49	Reverse Fenwick tree: range maximum and point increase	61
49.1	Problem	61
49.2	Reverse intervals	61
50	Fenwick tree of Fenwick trees	62

50.1	Motivation	62
50.2	Offline construction	62
51	Binary search tree: definition and operations	63
51.1	Definition	63
51.2	Operations without implementation details	63
51.3	Order-based queries	64
52	AVL tree: definition and depth bound	64
52.1	Definition	64
52.2	Minimum number of nodes of height h	64
53	Fixing imbalance in an AVL tree	65
53.1	Rotations	65
53.2	Four imbalance cases	65
54	AVL tree: find, insert, and erase	66
54.1	Find	66
54.2	Insertion	66
54.3	Deletion	67
54.4	Rebalance routine	67
55	Splay tree: definition and practical significance	68
55.1	Definition	68
55.2	Practical significance	68
55.3	Typical applications	68
56	Splay operations: zig, zig-zig, zig-zag, and splay	69
56.1	Primitive rotations	69
56.2	Zig	69
56.3	Zig-zig	69
56.4	Zig-zag	69
57	Potential method	70
57.1	Amortized analysis by potential	70
57.2	How to choose a potential	71
58	Amortized time of splay via the potential method	71
58.1	Rank potential	71
58.2	Access lemma	71
59	Splay tree: insert, erase, and find	72

59.1	Find	72
59.2	Insert	73
59.3	Erase	73
60	Splay tree: merge and split	74
60.1	Merge	74
60.2	Split	75
	Summary table	76
	Checklist of core ideas	76
61	Implicit-key search tree: sequence operations and range sums	77
61.1	Problem	77
61.2	Augmented node data	77
61.3	Split and merge by implicit position	78
61.4	Sequence operations	78
62	Hash functions, collisions, universal and k-independent families	79
62.1	Hashing model	79
62.2	Example: linear functions over a prime field	80
62.3	Example: polynomial hashing	80
63	Hash tables with chaining	81
63.1	Data structure	81
63.2	Operations	81
63.3	Expected time	81
63.4	Rehashing	82
64	Perfect hashing	82
64.1	Static dictionary problem	82
64.2	Two-level construction	82
65	Hash tables with open addressing	83
65.1	Data structure	83
65.2	Operations with tombstones	83
65.3	Rehashing	84
65.4	Expected-time statements	84
66	Bloom filter	84
66.1	Problem	84
66.2	Algorithm	85

66.3	False-positive probability	85
67	Cuckoo filter	86
67.1	Idea	86
67.2	Operations	86
67.3	Properties	86
68	Skip List	87
68.1	Definition	87
68.2	Search	87
68.3	Insert and erase	87
69	Cartesian tree and linear construction for sorted keys	88
69.1	Definition	88
69.2	Linear construction when keys are sorted	88
70	Merge and split in a Cartesian tree	89
70.1	Merge	89
70.2	Split by explicit key	89
71	Insert and erase in a Cartesian tree via merge and split	90
71.1	Insert	90
71.2	Erase	90
72	B-tree: definition, practical significance, and find	90
72.1	Definition	90
72.2	Practical significance	91
72.3	Find	91
73	Depth of a B-tree with n keys	91
74	B-tree insertion	92
74.1	Core idea	92
74.2	Split child	92
74.3	Insertion algorithm	92
75	B-tree deletion	93
75.1	Core idea	93
75.2	Cases	93
75.3	Borrow and merge	94
76	Red-black tree: definition	94
76.1	Definition	94

77	Depth of a red-black tree	95
78	Red-black tree insertion	95
78.1	BST insertion plus fix-up	95
78.2	Fix-up cases	96
79	Red-black tree deletion	96
79.1	BST deletion plus fix-up	96
79.2	Fix-up cases	96
80	Comparative analysis of search-tree implementations	97
80.1	Comparison table	97
80.2	Conceptual comparison	97
80.3	Which tree should one choose?	98

1 The RAM model and asymptotic running time

1.1 Motivation

Algorithms are usually analyzed independently of a particular programming language and a particular computer. The purpose of a machine model is to specify which elementary operations are counted as one step. The most common first model is the RAM model, short for *Random Access Machine*.

Definition 1.1 (RAM model). A RAM model consists of an unbounded array of memory cells, each storing one machine word, and a processor that can read or write any memory cell by its address. In the unit-cost word-RAM model, each elementary operation on one word costs $\mathcal{O}(1)$ time.

Typical unit-cost operations are:

- arithmetic on word-sized integers: addition, subtraction, multiplication in the simplified model;
- comparisons such as $x < y$, $x = y$;
- Boolean operations;
- reading and writing an array cell $A[i]$;
- pointer dereference and pointer assignment;
- loop counter increments and assignments.

Remark 1.1 (Word-size convention). If the input size is n , one usually assumes that an index into the input fits into one word, so a word contains $\Theta(\log n)$ bits. This is why array indexing is treated as constant time. If the integers are arbitrarily large, arithmetic is not automatically constant time; then bit-complexity analysis is needed.

1.2 Running time

Definition 1.2 (Running time). The running time $T(n)$ of an algorithm is the maximum number of elementary RAM operations performed on any input of size n , unless another convention such as average-case or expected running time is explicitly stated.

Example 1.1 (Linear scan). Consider the problem of checking whether an array contains a value x .

```
CONTAINS(A, x):  
  for i = 0 to n-1:  
    if A[i] == x:  
      return true  
  return false
```

In the worst case, either x is absent or it appears only at the last position. The loop performs n comparisons, hence $T(n) = \Theta(n)$.

Mental model. Do not try to count every CPU instruction. Count how many times the dominant elementary operation is executed. For a scan, the dominant operation is inspection of an array element. For comparison sorting, it is usually a comparison.

Common pitfalls. The RAM model does not say that every mathematical operation is constant time. It says that operations on one word are constant time. Multiplication of two thousand-digit integers is not a unit operation in bit-complexity.

Exercise 1.1. For the algorithm that computes the maximum element of an array of length n , write the exact number of comparisons and its asymptotic order.

2 Asymptotic notation: O , Ω , Θ

2.1 Definitions

Definition 2.1 (Big- O). For functions $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, we write

$$f(n) = \mathcal{O}(g(n))$$

if there exist constants $C > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq Cg(n) \quad \text{for all } n \geq n_0.$$

Definition 2.2 (Big- Ω). We write $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \geq cg(n) \quad \text{for all } n \geq n_0.$$

Definition 2.3 (Big- Θ). We write $f(n) = \Theta(g(n))$ if both $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$. Equivalently, there exist constants $c, C > 0$ and n_0 such that

$$cg(n) \leq f(n) \leq Cg(n) \quad \text{for all } n \geq n_0.$$

2.2 Independence of the starting index

Proposition 2.1 (Changing the starting index does not matter). Assume $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. If there exist constants $C > 0$ and N such that $f(n) \leq Cg(n)$ for all $n \geq N$, then the same asymptotic statement is unaffected by changing N to any larger threshold. In particular, finitely many exceptional small values of n do not affect $\mathcal{O}$, Ω , or Θ .

Proof. The definition already requires the inequality only for all sufficiently large n . If an inequality fails for finitely many small values, choose n_0 larger than all of them. The constants are meant to control the tail of the function, not its initial segment. $\square$

Example 2.1. Let $f(n) = 3n^2 + 10n + 7$. Then $f(n) = \Theta(n^2)$. For $n \geq 1$,

$$3n^2 \leq 3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2.$$

Thus $3n^2 \leq f(n) \leq 20n^2$ for $n \geq 1$.

Example 2.2. $\log n = \mathcal{O}(n^\varepsilon)$ for every fixed $\varepsilon > 0$, and $n^a = \mathcal{O}(n^b)$ whenever $a \leq b$. Exponential functions dominate polynomial functions: for fixed $a > 0$ and $b > 1$, $n^a = \mathcal{O}(b^n)$.

Common pitfalls. Writing $f(n) \leq \mathcal{O}(g(n))$ is informal and usually avoided in rigorous writing. Write $f(n) = \mathcal{O}(g(n))$. Also, $\mathcal{O}(g(n))$ is a set of functions, not a number.

Exercise 2.1. Prove that $5n \log n + 100n = \Theta(n \log n)$ for $n \geq 2$.

3 Range-sum queries in a static array: prefix sums

3.1 Problem

Given a static array $A[0], \dots, A[n-1]$, answer many queries of the form

$$\text{sum}(l, r) = A[l] + A[l+1] + \dots + A[r]$$

for $0 \leq l \leq r < n$.

The naive method scans the interval and costs $\Theta(r-l+1)$ per query. Prefix sums move this work into preprocessing.

Definition 3.1 (Prefix sums). Define an auxiliary array $P[0], \dots, P[n]$ by

$$P[0] = 0, \quad P[i+1] = P[i] + A[i].$$

Equivalently,

$$P[i] = \sum_{j=0}^{i-1} A[j].$$

Proposition 3.1 (Range-sum formula). *For every $0 \leq l \leq r < n$,*

$$A[l] + \dots + A[r] = P[r+1] - P[l].$$

Proof. By definition,

$$P[r+1] = A[0] + \dots + A[l-1] + A[l] + \dots + A[r]$$

and

$$P[l] = A[0] + \dots + A[l-1].$$

Subtracting cancels the common prefix and leaves exactly $A[l] + \dots + A[r]$. $\square$

```
BUILD-PREFIX(A):
    P[0] = 0
    for i = 0 to n-1:
        P[i+1] = P[i] + A[i]
    return P

RANGE-SUM(P, l, r):
    return P[r+1] - P[l]
```

3.2 Complexity

Preprocessing takes $\Theta(n)$ time and $\Theta(n)$ extra memory. Each query takes $\Theta(1)$ time.

Mental model. A prefix sum is a coordinate system for sums. Instead of summing an interval directly, represent it as the difference of two prefixes.

Common pitfalls. The cleanest convention is P of length $n+1$ with $P[0] = 0$. It eliminates a separate case for intervals starting at index 0.

4 Membership in a sorted array: binary search

4.1 Problem

Given a sorted array $A[0] \leq A[1] \leq \dots \leq A[n-1]$ and a value x , determine whether x occurs in A .

4.2 Core idea

At each step inspect the middle element. If $A[m] < x$, then all elements at indices $\leq m$ are too small. If $A[m] > x$, then all elements at indices $\geq m$ are too large. Thus one comparison discards roughly half of the search interval.

```
BINARY-SEARCH(A, x):
    l = 0
    r = n                # active interval is [l, r)
    while l < r:
        m = floor((l + r) / 2)
        if A[m] == x:
            return true
        else if A[m] < x:
            l = m + 1
        else:
            r = m
    return false
```

Theorem 4.1 (Correctness of binary search). *The algorithm returns true if and only if x occurs in the sorted array A .*

Proof. Use the loop invariant:

x can occur in A only inside the active interval $[l, r)$.

Initially, $[l, r) = [0, n)$, so the invariant is true. Suppose it holds before an iteration and let $m = \lfloor (l + r)/2 \rfloor$.

If $A[m] = x$, the algorithm correctly returns true. If $A[m] < x$, then by sortedness all elements $A[0], \dots, A[m]$ are $< x$, so x cannot occur there; updating $l = m + 1$ preserves the invariant. If $A[m] > x$, then all elements $A[m], \dots, A[n-1]$ are $> x$, so updating $r = m$ preserves the invariant.

The interval length $r - l$ strictly decreases each iteration, so the algorithm terminates. If it terminates with $l = r$, the active interval is empty. By the invariant, x cannot occur anywhere in A , so returning false is correct. $\square$

Proposition 4.1 (Running time). *Binary search runs in $\Theta(\log n)$ time and $\Theta(1)$ extra space.*

Proof. After each unsuccessful iteration, the active interval size is at most half of the previous size, up to rounding. After t iterations its size is at most $n/2^t$. Once $2^t > n$, the interval is empty. Hence $t = \mathcal{O}(\log n)$. A matching lower bound for this algorithm is obtained by considering an unsuccessful search: it must perform one comparison per halving until the interval becomes empty, hence $\Omega(\log n)$ iterations in the worst case. $\square$

Common pitfalls. The most common errors are off-by-one mistakes and infinite loops. The half-open convention $[l, r)$ is robust: the left boundary is included, the right boundary is excluded, and the empty interval is exactly $l = r$.

5 The sorting problem: two formulations and equivalence

5.1 Two formulations

Definition 5.1 (Sorting values). Given a sequence of keys $A = (a_1, \dots, a_n)$ from a totally ordered set, output a permutation of these keys in nondecreasing order:

$$b_1 \leq b_2 \leq \dots \leq b_n.$$

Definition 5.2 (Sorting records by key). Given records $R_1, \dots, R_n$ with keys $\text{key}(R_i)$, output the records in nondecreasing order of their keys.

The second formulation is the one used in databases and programming: the item has data attached to the key.

Definition 5.3 (Stable sorting). A sorting algorithm is stable if equal-key records preserve their original relative order. That is, if $i < j$ and $\text{key}(R_i) = \text{key}(R_j)$, then R_i appears before R_j in the output.

5.2 Equivalence

Proposition 5.1 (Equivalence of formulations). *Sorting records by key reduces to sorting pairs, and sorting values is the special case where each record is its own key. Therefore the two formulations are equivalent up to the representation of input items.*

Proof. A value a_i can be viewed as a record whose key is a_i . Hence record sorting generalizes value sorting.

Conversely, a record R_i can be represented as a pair $(\text{key}(R_i), i)$ plus a pointer to the original data. Sorting these pairs lexicographically by key and, if stability is required, by original index i , gives the records in sorted order and preserves the necessary identity of each item. Thus sorting values/pairs is enough to implement record sorting. $\square$

Mental model. A sorting algorithm does not merely rearrange numbers. It rearranges objects according to keys. Stability matters exactly when equal keys still have meaningful identities.

6 The formula $\log(n!) = \Theta(n \log n)$

Theorem 6.1. *For every logarithm base greater than 1,*

$$\log(n!) = \Theta(n \log n).$$

Proof. For the upper bound,

$$n! = 1 \cdot 2 \cdots n \leq n \cdot n \cdots n = n^n.$$

Taking logarithms gives

$$\log(n!) \leq \log(n^n) = n \log n,$$

so $\log(n!) = \mathcal{O}(n \log n)$.

For the lower bound, consider the last half of the factors:

$$n! = 1 \cdot 2 \cdots n \geq \left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) \cdots n.$$

There are at least $n/2$ factors, each at least $n/2$. Hence

$$n! \geq \left(\frac{n}{2}\right)^{n/2}$$

for all $n \geq 2$, up to harmless floor effects. Taking logarithms,

$$\log(n!) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2} \log n - \frac{n}{2} \log 2.$$

For $n \geq 4$, $\log n - \log 2 \geq \frac{1}{2} \log n$, hence

$$\log(n!) \geq \frac{n}{4} \log n.$$

Thus $\log(n!) = \Omega(n \log n)$, and the two bounds imply $\Theta(n \log n)$. $\square$

Remark 6.1. Changing the base of the logarithm only multiplies the logarithm by a positive constant. Therefore all standard logarithm bases give the same Θ -class.

7 Lower bound for comparison sorting

7.1 Comparison model

A comparison sorting algorithm obtains information about the input only by comparisons of the form $a_i < a_j$, $a_i \leq a_j$, or equivalent binary tests between keys. It may move items and use memory, but the ordering information comes only from comparisons.

7.2 Decision-tree argument

Assume the input keys are all distinct. Then every possible input order corresponds to a different permutation, and there are $n!$ possible sorted orders.

A deterministic comparison sort can be represented by a binary decision tree:

- each internal node is a comparison;
- each branch is one of the two comparison outcomes;
- each leaf is an output permutation.

Theorem 7.1 (Comparison-sorting lower bound). *Every deterministic comparison sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.*

Proof. For correctness on all inputs with distinct keys, the decision tree must have at least $n!$ leaves, because different input permutations require different output permutations. A binary tree of height h has at most 2^h leaves. Therefore

$$2^h \geq n!.$$

Taking logarithms gives

$$h \geq \log(n!) = \Omega(n \log n).$$

The height of the decision tree is exactly the worst-case number of comparisons. Hence every comparison sort has worst-case complexity $\Omega(n \log n)$. $\square$

Corollary 7.1. *Merge Sort, Heap Sort, and deterministic balanced Quick Sort are asymptotically optimal among comparison sorts in the worst case.*

Common pitfalls. The lower bound applies to comparison sorting. It does not contradict Counting Sort or Radix Sort, because these algorithms use more information about the keys than pairwise comparisons.

8 Merge Sort

8.1 Core idea

Merge Sort uses divide and conquer:

1. split the array into two halves;
2. recursively sort both halves;
3. merge the two sorted halves in linear time.

```

MERGE-SORT(A, l, r):           # sort A[l:r]
    if r - l <= 1:
        return
    m = floor((l + r) / 2)
    MERGE-SORT(A, l, m)
    MERGE-SORT(A, m, r)
    MERGE(A, l, m, r)

```

```

MERGE(A, l, m, r):
    i = l
    j = m
    B = empty array
    while i < m and j < r:
        if A[i] <= A[j]:
            append A[i] to B
            i = i + 1
        else:
            append A[j] to B
            j = j + 1
    append remaining A[i:m] to B
    append remaining A[j:r] to B
    copy B back into A[l:r]

```

Lemma 8.1 (Correctness of merge). *If $A[l : m)$ and $A[m : r)$ are sorted, then after Merge the segment $A[l : r)$ is sorted and contains exactly the same multiset of elements as before.*

Proof. At each step the smallest not-yet-output element among the two sorted lists is the smaller of their first remaining elements. The algorithm outputs exactly that element and advances the corresponding pointer. This preserves sortedness of the

output prefix. When one side is exhausted, the remaining elements of the other side are already sorted and at least as large as the last output element. Since every element is copied exactly once, the multiset is preserved. $\square$

Theorem 8.1 (Correctness of Merge Sort). *Merge-Sort outputs the input elements in nondecreasing order.*

Proof. Use induction on $n = r - l$. If $n \leq 1$, the segment is already sorted. For $n > 1$, the recursive calls sort $A[l : m)$ and $A[m : r)$ by the induction hypothesis. The merge lemma then implies that $A[l : r)$ becomes sorted and contains exactly the original elements. $\square$

Proposition 8.1 (Running time). *Merge Sort runs in $\Theta(n \log n)$ time and uses $\Theta(n)$ auxiliary memory in the standard implementation.*

Proof. The running time satisfies

$$T(n) = 2T(n/2) + \Theta(n),$$

where the linear term is the merge cost. By the recurrence analysis in the next section, $T(n) = \Theta(n \log n)$. The merge buffer may contain $\Theta(n)$ elements at the top level, hence the auxiliary memory is $\Theta(n)$. $\square$

Remark 8.1 (Stability). The merge procedure above uses $A[i] \leq A[j]$ and takes from the left half when keys are equal. Therefore Merge Sort is stable.

9 Solving $T(n) = 2T(n/2) + O(n)$

9.1 Recursion-tree proof

Assume for simplicity that $n = 2^k$. Suppose

$$T(n) \leq 2T(n/2) + cn$$

and $T(1) = O(1)$. At level 0 of the recursion tree, the total nonrecursive work is cn . At level 1, there are two subproblems of size $n/2$, so the total work is

$$2 \cdot c(n/2) = cn.$$

At level i , there are 2^i subproblems of size $n/2^i$, so the total work is

$$2^i \cdot c(n/2^i) = cn.$$

There are $\log_2 n$ non-leaf levels, and the leaves contribute $O(n)$ total work. Therefore

$$T(n) = O(n \log n).$$

If instead

$$T(n) \geq 2T(n/2) + cn$$

for some $c > 0$, the same level-sum argument gives

$$T(n) = \Omega(n \log n).$$

Thus if $T(n) = 2T(n/2) + \Theta(n)$, then

$$T(n) = \Theta(n \log n).$$

Theorem 9.1. *If $T(n) = 2T(n/2) + \Theta(n)$ and $T(1) = \Theta(1)$, then $T(n) = \Theta(n \log n)$.*

Mental model. The recursion has logarithmically many levels, and every level costs linear total work. Therefore total work is linear times logarithmic.

10 Counting inversions in an array

Definition 10.1 (Inversion). An inversion in an array $A[0 : n)$ is a pair of indices (i, j) such that

$$0 \leq i < j < n \quad \text{and} \quad A[i] > A[j].$$

The number of inversions measures how far the array is from being sorted.

10.1 Naive algorithm

Check all pairs (i, j) with $i < j$. This takes $\Theta(n^2)$ time.

10.2 Divide-and-conquer algorithm

Use Merge Sort. Count inversions in the left half, in the right half, and across the two halves.

If both halves are sorted, cross inversions can be counted during merge. When $A[i] > A[j]$ with i in the left half and j in the right half, then all remaining elements $A[i], A[i+1], \dots, A[m-1]$ in the left half are also $> A[j]$. Thus this single comparison contributes $m - i$ cross inversions.

```

COUNT-INV(A, l, r):
    if r - l <= 1:
        return 0
    m = floor((l + r) / 2)
    ans = COUNT-INV(A, l, m) + COUNT-INV(A, m, r)
    ans += MERGE-AND-COUNT(A, l, m, r)
    return ans

```

Theorem 10.1. *The merge-based algorithm counts all inversions correctly in $\Theta(n \log n)$ time.*

Proof. Every inversion belongs to exactly one of three classes: both indices in the left half, both in the right half, or one in each half. Recursive calls count the first two classes. During merge, if the current right element is smaller than the current left element, then it forms inversions with all remaining left elements, and with no earlier left elements because those have already been output and are at most the right element. Thus the merge step counts exactly the cross inversions. The running time satisfies the Merge Sort recurrence $T(n) = 2T(n/2) + \Theta(n)$, hence $T(n) = \Theta(n \log n)$. $\square$

11 Quick Sort

11.1 Core idea

Quick Sort chooses a pivot, partitions the array into elements smaller than the pivot and elements larger than the pivot, and recursively sorts the two sides.

```

QUICKSORT(A, l, r):                # sort A[l:r)
    if r - l <= 1:
        return

```

```

p = CHOOSE-PIVOT(A, l, r)
q = PARTITION(A, l, r, p)
QUICKSORT(A, l, q)
QUICKSORT(A, q+1, r)

```

Here q is the final position of the pivot after partition.

11.2 Partition invariant

A standard Lomuto-style partition maintains:

$$A[l : i] < p, \quad A[i : j] \text{ not yet classified}, \quad A[j : r - 1] \geq p,$$

or an equivalent invariant depending on the implementation. The essential point is that partitioning takes linear time and places the pivot into its final sorted position.

Proposition 11.1 (Worst-case time). *Quick Sort has worst-case running time $\Theta(n^2)$.*

Proof. If the pivot is always the smallest or largest element, the recursive subproblem sizes are 0 and $n - 1$. Then

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

The upper bound follows because the sum $n + (n - 1) + \dots + 1$ is $\Theta(n^2)$. □

Proposition 11.2 (Expected time with a random pivot). *Randomized Quick Sort runs in expected time $\Theta(n \log n)$.*

Proof sketch. The cleanest proof counts comparisons. For two elements of ranks $i < j$ in the sorted order, they are compared if and only if the first pivot chosen among the elements with ranks $i, i + 1, \dots, j$ is one of these two elements. This happens with probability

$$\frac{2}{j - i + 1}.$$

Therefore the expected number of comparisons is

$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = \sum_{d=1}^{n-1} (n - d) \frac{2}{d + 1} = O(n \log n).$$

Since partitioning work is linear in the number of comparisons up to constants, the expected running time is $O(n \log n)$. The lower bound $\Omega(n \log n)$ follows from comparison-sorting optimality for algorithms that sort by comparisons. □

Common pitfalls. Quick Sort is not automatically stable. Also, many simple two-way partitions behave poorly on arrays with many equal keys. In practice, three-way partitioning into $< p$, $= p$, and $> p$ is often preferable.

12 Randomized Quick Select

12.1 Problem

Given an array $A[0 : n)$ and an integer k , find the element of rank k in sorted order. We use zero-based ranks, so $k = 0$ means the minimum and $k = n - 1$ means the maximum.

12.2 Core idea

Quick Select is like Quick Sort, but after partitioning it recurses into only one side: the side that contains the desired rank.

```

QUICKSELECT(A, l, r, k):          # k is an absolute rank in A
    p = random pivot from A[l:r]
    q = PARTITION(A, l, r, p)
    if k == q:
        return A[q]
    else if k < q:
        return QUICKSELECT(A, l, q, k)
    else:
        return QUICKSELECT(A, q+1, r, k)

```

Theorem 12.1 (Correctness). *Quick Select returns the element of rank k .*

Proof. After partition, the pivot is in the same position it would occupy in the fully sorted array: all elements to its left are at most it, and all elements to its right are at least it. If $q = k$, the pivot is exactly the desired element. If $k < q$, the desired element must lie in the left part. If $k > q$, it must lie in the right part. The recursive call therefore preserves the target. Termination follows because the recursive interval strictly decreases. $\square$

Proposition 12.1 (Expected running time). *Randomized Quick Select runs in expected $O(n)$ time.*

Proof sketch. Call a pivot good if its rank lies between the 25th and 75th percentiles of the current subarray. A uniformly random pivot is good with probability at least $1/2$. If the pivot is good, the next recursive subproblem has size at most $3n/4$. Each partition costs linear time in the current size. The expected number of trials until a good pivot is constant. Hence the expected cost to reduce size from n to at most $3n/4$ is $O(n)$. Summing over phases gives

$$O(n) + O(3n/4) + O((3/4)^2n) + \dots = O(n).$$

$\square$

Remark 12.1. The worst-case time is $\Theta(n^2)$ if pivots are consistently terrible. The expected bound is over the internal random choices of the algorithm.

13 Deterministic linear-time selection

13.1 Median of medians

The deterministic selection algorithm chooses a pivot that is guaranteed to be sufficiently balanced.

```

SELECT(A, k):
    if n <= constant:
        sort A directly and return A[k]

    split A into groups of 5 elements

```

```

sort each group and extract its median
p = SELECT(list of medians, floor(number_of_groups / 2))
partition A into L = elements < p, E = elements = p, G = elements > p

if k < |L|:
    return SELECT(L, k)
else if k < |L| + |E|:
    return p
else:
    return SELECT(G, k - |L| - |E|)

```

13.2 Why the pivot is good

Let there be approximately $n/5$ groups. At least half of the group medians are at least the median of medians p . In each such group, at least three elements are at least that group median, hence at least p . Ignoring a constant number of incomplete groups, at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p . Similarly, at least $3n/10$ elements are at most p .

Therefore, after partition, the recursive side has size at most $7n/10 + O(1)$.

Theorem 13.1 (Deterministic selection). *The median-of-medians algorithm finds the k th order statistic in worst-case $O(n)$ time.*

Proof. The algorithm first recursively selects the median of approximately $n/5$ medians, then recurses on at most $7n/10 + O(1)$ elements. Group sorting and partitioning cost $O(n)$. Hence

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + cn.$$

We prove by induction that $T(n) \leq an$ for a sufficiently large constant a . Then

$$T(n) \leq a \frac{n}{5} + a \frac{7n}{10} + cn + O(1) = a \frac{9n}{10} + cn + O(1).$$

Choose a large enough so that $a/10 \geq c$ and absorb the $O(1)$ term into the base cases. Then $T(n) \leq an$. Therefore $T(n) = O(n)$. $\square$

Mental model. Randomized selection hopes for a good pivot. Median-of-medians manufactures a pivot that is never too close to either extreme.

14 Deterministic Quick Sort in $O(n \log n)$

14.1 Core idea

Use deterministic linear-time selection to choose an exact median as the pivot at every recursive call. Then the partition is perfectly balanced up to one element.

```

DETERMINISTIC-QUICKSORT(A, l, r):
    if r - l <= 1:
        return
    p = SELECT(A[l:r], floor((r-l)/2))
    q = PARTITION-AROUND-VALUE(A, l, r, p)

```

```

DETERMINISTIC-QUICKSORT(A, l, q)
DETERMINISTIC-QUICKSORT(A, q+1, r)

```

Theorem 14.1. *Deterministic Quick Sort with exact median pivots runs in worst-case $O(n \log n)$ time.*

Proof. At a subproblem of size n , selecting the exact median costs $O(n)$ and partitioning costs $O(n)$. The two recursive subproblems have sizes at most $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil - 1$. Therefore

$$T(n) \leq T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil - 1) + cn.$$

This is bounded by

$$T(n) \leq 2T(n/2) + cn,$$

up to rounding, and hence $T(n) = O(n \log n)$ by the Merge Sort recurrence. Since every comparison sort needs $\Omega(n \log n)$ comparisons in the worst case, the asymptotic order is optimal in the comparison model. $\square$

Remark 14.1. This algorithm is mainly theoretical. In practice, computing exact medians at every level introduces large constants. Randomized Quick Sort or introsort is usually preferred.

15 Stable Counting Sort and sorting pairs

15.1 Problem

Counting Sort sorts keys from a small integer range. Assume keys lie in $\{0, 1, \dots, K-1\}$.

```

COUNTING-SORT(A, K):
    C[0:K] = all zeros
    for x in A:
        C[key(x)] = C[key(x)] + 1

    for i = 1 to K-1:
        C[i] = C[i] + C[i-1]

    B[0:n] = empty
    for i = n-1 downto 0:
        x = A[i]
        C[key(x)] = C[key(x)] - 1
        B[C[key(x)]] = x

    return B

```

15.2 Correctness and stability

After the prefix accumulation, $C[t]$ equals the number of elements with key at most t . Thus the final block of key t in the output ends at index $C[t] - 1$.

Processing the input from right to left makes the algorithm stable: among equal keys, the later input element is placed later in the output, so the earlier input element remains earlier.

Theorem 15.1. *Stable Counting Sort sorts n records with integer keys in $\{0, \dots, K-1\}$ in $\Theta(n+K)$ time and $\Theta(n+K)$ space.*

Proof. The first loop counts n records. The second loop scans K counters. The final placement loop scans n records. Therefore the time is $\Theta(n+K)$. The output array uses $\Theta(n)$ space and the count array uses $\Theta(K)$ space. Correctness follows from placing each key into its reserved block. Stability follows from the right-to-left traversal. $\square$

15.3 Sorting pairs

To sort pairs (a, b) lexicographically by (a, b) using stable sorts:

1. stable-sort by the secondary key b ;
2. stable-sort by the primary key a .

The second stable sort groups equal a values while preserving the previous order by b inside each group.

Proposition 15.1. *Two stable sorts, first by secondary key and then by primary key, produce lexicographic order by pair.*

Proof. After the final sort, primary keys are nondecreasing. Consider two records with equal primary key. Stability of the final sort preserves their relative order from the previous pass. Since the previous pass sorted by secondary key, they are ordered by secondary key inside the equal-primary block. $\square$

16 LSD Radix Sort

16.1 Core idea

LSD means *least significant digit*. To sort fixed-length digit strings, sort by the last digit, then by the previous digit, and so on, using a stable sorting algorithm at each digit.

For example, to sort decimal numbers with exactly d digits, perform stable Counting Sort by digit $d-1$, then digit $d-2$, and continue down to digit 0.

```
LSD-RADIX-SORT(A, d, base):
  for pos = d-1 downto 0:
    A = STABLE-COUNTING-SORT(A, key = digit(pos), K = base)
  return A
```

Theorem 16.1 (Correctness of LSD Radix Sort). *After processing positions $d-1, d-2, \dots, t$, the array is sorted by the suffix of digits from position t to position $d-1$. Hence after the final pass it is sorted by the whole key.*

Proof. Use backward induction on t . After sorting by the last digit, the array is sorted by the suffix of length 1. Suppose the array is sorted by the suffix starting at $t+1$. Stable sorting by digit t puts records in nondecreasing order of digit t . Among records with equal digit t , stability preserves their previous order, which by the induction hypothesis is sorted by the suffix $t+1, \dots, d-1$. Therefore the array becomes sorted by the longer suffix $t, \dots, d-1$. $\square$

Proposition 16.1 (Complexity). *If each stable digit sort costs $\Theta(n + K)$ and there are d digits, LSD Radix Sort runs in*

$$\Theta(d(n + K))$$

time.

Common pitfalls. Stability is essential. Without stability, later digit passes can destroy the ordering established by less significant digits.

17 Singly and doubly linked lists

17.1 Singly linked list

A singly linked list consists of nodes. Each node stores a value and a pointer to the next node.

```
Node:
  value
  next
```

If we have a pointer to a node, insertion after it is constant time.

```
INSERT-AFTER(x, y):      # insert node y after node x
  y.next = x.next
  x.next = y
```

Deletion after a node is also constant time.

```
DELETE-AFTER(x):
  y = x.next
  x.next = y.next
  return y
```

However, deleting a given node requires knowing its predecessor, which may require a scan.

17.2 Doubly linked list

A doubly linked list stores both next and previous pointers.

```
DNode:
  value
  prev
  next
```

Given a pointer to a node, deletion is constant time.

```
DELETE(x):
  x.prev.next = x.next
  x.next.prev = x.prev
```


Sentinel nodes are often used to avoid boundary cases at the head and tail.

Operation	Singly linked list	Doubly linked list
Insert after known node	$O(1)$	$O(1)$
Delete after known node	$O(1)$	$O(1)$
Delete known node	needs predecessor	$O(1)$
Move backward	not supported directly	$O(1)$
Extra pointer per node	one	two

Common pitfalls. A linked list gives constant-time insertion/deletion only when the relevant node or predecessor is already known. Searching for a position is still linear.

18 Stack data structure: list implementation

Definition 18.1 (Stack). A stack is an abstract data type with last-in-first-out behavior. It supports:

- $\text{Push}(x)$: add x to the top;
- $\text{Pop}()$: remove and return the top element;
- $\text{Top}()$: return the top without removing it;
- $\text{Empty}()$: test whether the stack is empty.

18.1 Implementation by a singly linked list

Use the head of the list as the top of the stack.

```
PUSH(S, x):
    node = new Node(x)
    node.next = S.head
    S.head = node

POP(S):
    if S.head == null:
        error "empty stack"
    node = S.head
    S.head = S.head.next
    return node.value
```

Proposition 18.1. A linked-list stack supports Push, Pop, Top, and Empty in $O(1)$ worst-case time.

Proof. Each operation changes or reads only the head pointer and possibly one node. No traversal is performed. Therefore each operation costs constant time. $\square$

19 Checking bracket sequences with several bracket types

19.1 Problem

Given a string over brackets such as

(), [], {},

determine whether it is a correct bracket sequence.

19.2 Algorithm

Use a stack of opening brackets.

```
CHECK-BRACKETS(s):
  S = empty stack
  for c in s:
    if c is an opening bracket:
      PUSH(S, c)
    else if c is a closing bracket:
      if EMPTY(S):
        return false
      x = POP(S)
      if x does not match c:
        return false
  return EMPTY(S)
```

Theorem 19.1. *The stack algorithm accepts exactly the correct bracket sequences.*

Proof. The invariant after reading a prefix is: the stack contains exactly the opening brackets from that prefix that have not yet been matched, in their nesting order; the top is the most recent unmatched opening bracket.

When an opening bracket is read, it becomes unmatched and is pushed. When a closing bracket is read, it must match the most recent unmatched opening bracket; otherwise the nesting structure is impossible. Thus popping and checking the top is necessary and sufficient at that step. At the end, the whole string is correct exactly when no unmatched opening brackets remain, i.e. the stack is empty. $\square$

Proposition 19.1. *The algorithm runs in $\Theta(n)$ time and uses $O(n)$ memory.*

20 Nearest smaller/greater elements in a static array

20.1 Problem

For each position i , find the nearest index to the left or right whose value is smaller or greater than $A[i]$.

Typical variants:

- nearest smaller to the left;
- nearest smaller to the right;
- nearest greater to the left;
- nearest greater to the right.

20.2 Nearest smaller to the left

Maintain a stack of candidate indices with strictly increasing values.

```

NEAREST-SMALLER-LEFT(A):
    S = empty stack of indices
    ans[0:n]
    for i = 0 to n-1:
        while not EMPTY(S) and A[TOP(S)] >= A[i]:
            POP(S)
        if EMPTY(S):
            ans[i] = -1
        else:
            ans[i] = TOP(S)
        PUSH(S, i)
    return ans

```

Theorem 20.1. *The algorithm returns for every i the nearest index $j < i$ such that $A[j] < A[i]$, or -1 if no such index exists.*

Proof. Before processing i , the stack contains indices to the left of i whose values are strictly increasing from bottom to top. Any index removed by the while-loop has value at least $A[i]$, so it cannot be the answer for i . It also cannot be the answer for any future position $t > i$ for which $A[t] \leq A[i]$, because i is closer and no larger. After all such indices are removed, the top of the stack, if it exists, has value $< A[i]$. It is the nearest such index because all indices above it were removed and all later surviving candidates would have been above it. Then pushing i restores the monotonic invariant. $\square$

Proposition 20.1 (Amortized complexity). *The monotonic-stack algorithm runs in $\Theta(n)$ time.*

Proof. Each index is pushed exactly once and popped at most once. Therefore the total number of stack operations is $O(n)$. $\square$

20.3 Other variants

To find nearest greater instead of smaller, reverse the comparison. To find nearest elements to the right, scan from right to left.

Variant	Scan direction	While condition
smaller left	left to right	pop while $A[\text{top}] \geq A[i]$
greater left	left to right	pop while $A[\text{top}] \leq A[i]$
smaller right	right to left	pop while $A[\text{top}] \geq A[i]$
greater right	right to left	pop while $A[\text{top}] \leq A[i]$

Common pitfalls. Strict and non-strict inequalities determine how equal values are treated. Decide whether you need strictly smaller, smaller-or-equal, strictly greater, or greater-or-equal before writing the while condition.

21 Evaluating expressions in Reverse Polish Notation

Definition 21.1 (Reverse Polish Notation). Reverse Polish Notation, or postfix notation, writes an operator after its operands. For example,

$$(2 + 3) \cdot 4$$

is written as

$$2\ 3\ +\ 4\ \cdot\ .$$

21.1 Algorithm

Scan tokens from left to right. Push numbers. When an operator appears, pop its operands, apply the operator, and push the result.

```
EVALUATE-RPN(tokens):
    S = empty stack
    for token in tokens:
        if token is a number:
            PUSH(S, token)
        else:
            b = POP(S)
            a = POP(S)
            PUSH(S, APPLY(token, a, b))
    if size(S) != 1:
        error "invalid expression"
    return POP(S)
```

Theorem 21.1. *For every valid Reverse Polish expression, the stack algorithm returns its value.*

Proof. The invariant after reading any prefix is: the stack contains the values of all complete subexpressions that have been formed by this prefix and not yet used as operands of a later operator, in their natural order.

A number token is a complete subexpression, so pushing it preserves the invariant. If an operator token is read, validity of RPN ensures that the two most recent available subexpressions are exactly its operands. Popping them, applying the operator, and pushing the result replaces two complete subexpressions by the complete subexpression they form together. At the end, a valid expression has exactly one complete expression, so the stack contains exactly one value: the answer. $\square$

Proposition 21.1. *Evaluation of an RPN expression with n tokens takes $\Theta(n)$ time and $O(n)$ stack space.*

Common pitfalls. For noncommutative operators, operand order matters. If the algorithm pops b first and then a , the operation is $a \text{ op } b$, not $b \text{ op } a$.

Part I summary table

Topic	Main complexity	Key idea
-------	-----------------	----------

RAM model	operation-dependent	word operations cost $O(1)$
Prefix sums	build $O(n)$, query $O(1)$	interval sum = difference of prefixes
Binary search	$O(\log n)$	discard half by sortedness
Comparison sorting lower bound	$\Omega(n \log n)$	decision tree has at least $n!$ leaves
Merge Sort	$\Theta(n \log n)$	sort halves and merge
Inversion counting	$\Theta(n \log n)$	count cross inversions during merge
Randomized Quick Sort	expected $\Theta(n \log n)$	random pivots give balanced recursion on average
Randomized Quick Select	expected $O(n)$	recurse into one side only
Median of medians	worst-case $O(n)$	deterministic good pivot
Deterministic median Quick Sort	worst-case $O(n \log n)$	exact median pivot at each level
Counting Sort	$\Theta(n + K)$	count key frequencies and place stably
LSD Radix Sort	$\Theta(d(n + K))$	stable digit passes from least significant to most significant
Linked-list stack	$O(1)$ per operation	top is the list head
Bracket checking	$\Theta(n)$	stack of unmatched opening brackets
Nearest smaller/greater	$\Theta(n)$	monotonic stack, each index pushed and popped once
RPN evaluation	$\Theta(n)$	stack of partial expression values

Part I practice problems

Exercise 21.1. Implement binary search using the closed interval convention $[l, r]$. State the loop invariant and prove termination.

Exercise 21.2. Modify Merge Sort to sort records stably by key. Explain exactly where stability is enforced.

Exercise 21.3. Given an array A , count pairs (i, j) such that $i < j$ and $A[i] \geq A[j]$. How must the merge-counting condition change?

Exercise 21.4. Show that if Quick Sort always splits into subproblems of sizes at most αn and $(1 - \alpha)n$ for a fixed $0 < \alpha < 1$, then its running time is $O(n \log n)$.

Exercise 21.5. Use stable Counting Sort to sort triples (a, b, c) lexicographically. In which order should the stable passes be applied?

Exercise 21.6. For each position in an array, find the nearest index to the left with value less than or equal to the current value. Write the monotonic-stack condition carefully.

Exercise 21.7. Extend the RPN evaluator to support unary minus. How does this affect validation of the number of operands?

22 Supporting the minimum in a stack

22.1 Problem

A stack supports three basic operations:

- `push(x)`: add an element on top;
- `pop()`: remove the top element;
- `top()`: return the top element.

The goal is to add `getMin()` returning the minimum element currently stored in the stack, while keeping all operations in $\mathcal{O}(1)$ worst-case time.

Mental model. A stack forgets everything except the top. To remember the current minimum after arbitrary pops, each element must carry enough information about the stack prefix below it.

22.2 Augmented stack

Store pairs

$$(x, m), \quad m = \min\{\text{all elements in the stack after pushing } x\}.$$

Thus the second coordinate of the top pair is always the current minimum.

```
PUSH(x):
    if S.empty():
        S.push((x, x))
    else:
        S.push((x, min(x, S.top().min)))

POP():
    return S.pop().value

GET-MIN():
    return S.top().min
```

Theorem 22.1 (Correctness of the min-stack). *After every sequence of stack operations, if the stack is nonempty, the field `S.top().min` equals the minimum of all elements currently stored in the stack.*

Proof. We prove a stronger invariant: for every stored pair at depth i from the bottom, its second component is the minimum of the first i stack elements.

Initially the stack is empty, so the invariant holds vacuously. When pushing the first element x , the stored minimum is x , which is correct. If the stack is nonempty and the previous current minimum is m , then after pushing x the new minimum is exactly $\min(x, m)$. Therefore the pushed pair stores the correct prefix minimum, while all older pairs are unchanged and remain correct.

A pop removes only the top pair. All remaining pairs were correct before the pop and their represented stack prefixes have not changed. Hence the invariant is preserved. The top pair corresponds to the whole stack, so its stored minimum is the current minimum. $\square$

Proposition 22.1 (Complexity). *`push`, `pop`, `top`, and `getMin` all take $\Theta(1)$ worst-case time. The memory overhead is one extra value per element, hence $\Theta(n)$ memory for n elements.*

Common pitfalls. Do not store only one global variable `currentMin`. After popping the minimum, you would not know the previous minimum. The stored history of minima is what makes the structure correct.

Exercise 22.1. Modify the min-stack so that it also supports `getMax()` in $\mathcal{O}(1)$ worst-case time.

23 Queue data structure: linked-list implementation

23.1 Definition

A queue is a first-in-first-out data structure. It supports:

- `enqueue(x)`: add x to the back;
- `dequeue()`: remove and return the front element;
- `front()`: return the front element without removing it.

Definition 23.1 (FIFO order). A data structure has FIFO order if elements leave it in the same order in which they entered it.

23.2 Singly linked-list implementation

Store two pointers:

`head` = front node, `tail` = back node.

Each node stores a value and a pointer to the next node.

```

ENQUEUE(x):
    v = new Node(x)
    v.next = null
    if tail == null:
        head = v
        tail = v
    else:
        tail.next = v
        tail = v

DEQUEUE():
    if head == null: error "empty queue"
    x = head.value
    head = head.next
    if head == null:
        tail = null
    return x

FRONT():
    if head == null: error "empty queue"
    return head.value

```

Theorem 23.1 (Correctness). *The linked-list implementation returns elements in FIFO order.*

Proof. The invariant is that the linked list from `head` to `tail` stores exactly the queue contents from front to back.

Initially both pointers are null, so the invariant holds. `enqueue(x)` creates a new node after the old `tail`. Therefore x becomes the last element and all older elements keep

their order. If the queue was empty, both head and tail point to the unique node, which is correct.

dequeue() removes head, which is exactly the oldest currently stored element. The new head is the next node in order. If no node remains, tail is also set to null. Hence the invariant is preserved, and the removed value is the FIFO front. $\square$

Proposition 23.1 (Complexity). *With explicit head and tail pointers, enqueue, dequeue, and front take $\Theta(1)$ worst-case time.*

Common pitfalls. If you keep only a pointer to the head of a singly linked list, then appending at the back requires scanning the whole list. The tail pointer is essential for $\mathcal{O}(1)$ enqueue.

24 Queue on two stacks and minimum in a queue

24.1 Queue on two stacks

Use two stacks:

- in: receives newly enqueued elements;
- out: provides elements for dequeuing.

```

ENQUEUE(x):
    in.push(x)

MOVE-IF-NEEDED():
    if out.empty():
        while not in.empty():
            out.push(in.pop())

DEQUEUE():
    MOVE-IF-NEEDED()
    if out.empty(): error "empty queue"
    return out.pop()

FRONT():
    MOVE-IF-NEEDED()
    if out.empty(): error "empty queue"
    return out.top()

```

Mental model. The in stack stores the newest part in reverse chronological order. Moving it into out reverses it again, so the oldest of that block becomes accessible first.

Theorem 24.1 (Correctness). *The two-stack implementation realizes FIFO queue order.*

Proof. At any time, the logical queue consists of the elements of out from top to bottom, followed by the elements of in from bottom to top.

enqueue(x) pushes x on top of in; in the logical view this appends x at the back. If out is nonempty, dequeue() removes its top, which is the logical front. If out is empty, moving all elements from in to out reverses their order, so the oldest element among them becomes the top of out. Therefore the next pop removes the logical front. The invariant is preserved in every case. $\square$

24.2 Amortized complexity

Proposition 24.1 (Amortized cost). *Each queue operation has $\mathcal{O}(1)$ amortized time.*

Proof. Each element is pushed into `in` once, moved from `in` to `out` at most once, and popped from `out` once. Thus over any sequence of m operations, the total number of stack operations is $\mathcal{O}(m)$. Therefore the average cost per operation is $\mathcal{O}(1)$. $\square$

24.3 Minimum in a queue

Use two min-stacks instead of ordinary stacks. The queue minimum is

$$\min(\text{in.getMin()}, \text{out.getMin()}),$$

ignoring empty stacks.

```
GET-MIN():
    if in.empty() and out.empty(): error "empty queue"
    if in.empty(): return out.getMin()
    if out.empty(): return in.getMin()
    return min(in.getMin(), out.getMin())
```

Theorem 24.2. *The two-stack queue with min-stacks supports enqueue, dequeue, front, and getMin in $\mathcal{O}(1)$ amortized time.*

Proof. Correctness of queue order follows from the previous theorem. Correctness of `getMin` follows because every current queue element is stored in exactly one of the two stacks, and each min-stack returns the minimum of its own contents. The minimum of the union is the minimum of the two stack minima. Complexity follows from the amortized analysis above and the $\mathcal{O}(1)$ worst-case operations of min-stacks. $\square$

Common pitfalls. When moving elements from `in` to `out`, do not copy the stored minimum fields directly. Re-push values into the second min-stack so that its own prefix minima are recomputed in the new stack order.

25 Binary heap: definition and array representation

25.1 Complete binary tree

A binary heap is a complete binary tree stored implicitly in an array. Complete means that every level except possibly the last is full, and the last level is filled from left to right.

For zero-based indexing:

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2.$$

Definition 25.1 (Min-heap property). An array $A[0 \dots n-1]$ is a binary min-heap if for every valid child index j with parent p ,

$$A[p] \leq A[j].$$

Equivalently, every node stores a key not larger than the keys stored in its children.

Proposition 25.1 (Minimum at the root). *If A is a nonempty min-heap, then $A[0]$ is a minimum element of the whole heap.*

Proof. Take any node v . On the path from the root to v , each parent key is at most its child key. Therefore $A[0] \leq A[v]$. Since v was arbitrary, $A[0]$ is a global minimum. $\square$

Proposition 25.2 (Height). *A binary heap with n elements has height $\lfloor \log_2 n \rfloor$.*

Proof. A complete binary tree of height h has at least 2^h nodes and at most $2^{h+1} - 1$ nodes. Hence $2^h \leq n < 2^{h+1}$, so $h = \lfloor \log_2 n \rfloor$. $\square$

Mental model. The array order is not sorted order. It is level order. The heap guarantees only parent-child order, not left-to-right order on the same level.

26 siftUp and siftDown in a binary heap

26.1 siftUp

$\text{siftUp}(i)$ is used when only the element at index i may be too small compared with its parent. It repeatedly swaps the element with its parent while the heap property is violated.

```
SIFT-UP(i):
    while i > 0 and A[i] < A[parent(i)]:
        swap A[i], A[parent(i)]
        i = parent(i)
```

Theorem 26.1 (Correctness of siftUp). *Assume all heap edges satisfy the min-heap property except possibly the edge between index i and its parent. After $\text{siftUp}(i)$ terminates, the whole array satisfies the min-heap property.*

Proof. The only possible violation is that the current element is smaller than its parent. If this violation exists, swapping the current element with its parent fixes that edge. Could the swap break heap order with the children of the moved-down element? Before the swap, the parent key was at most all children keys except possibly the current child. After the swap, the larger key moves into the child's position. It is at most the old child key's children because those children were below the old child, and the old child subtree was a heap. Thus no violation is created below.

The possible violation moves one level upward, between the current element and its new parent. The loop repeats until either the element reaches the root or its parent is no larger than it. At that point there is no remaining violation. $\square$

26.2 siftDown

$\text{siftDown}(i)$ is used when the element at index i may be too large compared with its children, while the left and right subtrees are already heaps.

```
SIFT-DOWN(i):
    while left(i) < heapSize:
        j = left(i)
```

```

    if right(i) < heapSize and A[right(i)] < A[left(i)]:
        j = right(i)
    if A[i] <= A[j]:
        break
    swap A[i], A[j]
    i = j

```

Theorem 26.2 (Correctness of `siftDown`). *Assume the left and right subtrees of index i are min-heaps and the only possible heap violations are between i and its children. After `siftDown( $i$ )` terminates, the subtree rooted at the initial index i is a min-heap.*

Proof. At a current node i , choose the smaller child j . If $A[i] \leq A[j]$, then $A[i]$ is at most both children, because j is the smaller child. Since both child subtrees are heaps, the subtree rooted at i is a heap.

Otherwise $A[i] > A[j]$. Swapping $A[i]$ with the smaller child makes the new parent key equal to the smaller child key, hence it is at most the other child and at most the moved-down key. Therefore the heap property holds at the new parent. The only possible violation is now inside the child subtree where the larger key moved. Repeating the same argument moves the possible violation strictly downward. The loop must terminate at a leaf or at a node satisfying the heap property. $\square$

Proposition 26.1 (Complexity). *Both `siftUp` and `siftDown` take $\mathcal{O}(\log n)$ worst-case time in a heap of size n .*

27 Heap operations via `siftUp` and `siftDown`

27.1 Operations

A binary min-heap supports the following standard operations.

```

INSERT(x):
    A[heapSize] = x
    heapSize = heapSize + 1
    SIFT-UP(heapSize - 1)

GET-MIN():
    if heapSize == 0: error "empty heap"
    return A[0]

EXTRACT-MIN():
    if heapSize == 0: error "empty heap"
    ans = A[0]
    A[0] = A[heapSize - 1]
    heapSize = heapSize - 1
    if heapSize > 0:
        SIFT-DOWN(0)
    return ans

DECREASE-KEY(i, newKey):
    require newKey <= A[i]
    A[i] = newKey
    SIFT-UP(i)

```

Theorem 27.1 (Correctness). *The operations above preserve the min-heap property and have the specified behavior.*

Proof. insert first adds a new leaf. Only the edge between this leaf and its parent can violate the heap property, so siftUp restores the heap.

getMin is correct because the minimum is stored at the root.

extractMin removes the root, which is a minimum. The last element is moved to the root to preserve the complete-tree shape. The two child subtrees of the root remain heaps; only the root may violate heap order. Therefore siftDown(0) restores the heap.

decreaseKey can only make the modified key smaller. Thus it cannot violate heap order with its children; it may only violate heap order with its parent. Therefore siftUp restores the heap. $\square$

Operation	Worst-case time	Reason
getMin	$\Theta(1)$	read the root
insert	$\mathcal{O}(\log n)$	one siftUp
extractMin	$\mathcal{O}(\log n)$	one siftDown
decreaseKey	$\mathcal{O}(\log n)$	one siftUp

Common pitfalls. decreaseKey requires a way to locate the item in the heap, usually an index or a handle. Without such a pointer, finding the item may cost $\Theta(n)$.

28 Building a heap in linear time: heapify

28.1 Bottom-up heap construction

The naive method inserts elements one by one and costs $\mathcal{O}(n \log n)$. Floyd's heap construction is faster.

```
BUILD-HEAP(A):
    heapSize = n
    for i = floor(n/2) - 1 downto 0:
        SIFT-DOWN(i)
```

Indices $\lfloor n/2 \rfloor, \dots, n-1$ are leaves and are already heaps.

Theorem 28.1 (Correctness of heapify). *After BUILD-HEAP(A), the array A satisfies the min-heap property.*

Proof. We process vertices in reverse level order. The invariant is: before calling siftDown(i), all subtrees rooted at indices greater than i are already heaps. The children of i, if they exist, have indices greater than i, so their subtrees are heaps. Therefore the precondition of siftDown(i) holds, and after it finishes the subtree rooted at i is a heap. By descending induction on i, all subtrees become heaps, in particular the subtree rooted at 0, which is the whole heap. $\square$

Theorem 28.2 (Linear-time heap construction). *BUILD-HEAP runs in $\mathcal{O}(n)$ time.*

Proof. A node of height h can move down at most h levels. The number of nodes of height h in a complete binary tree is at most $\lceil n/2^{h+1} \rceil$. Hence the total work is bounded by

$$\sum_{h \geq 0} \mathcal{O}\left(\frac{n}{2^{h+1}} \cdot h\right) = \mathcal{O}\left(n \sum_{h \geq 0} \frac{h}{2^h}\right) = \mathcal{O}(n),$$

where the series converges. Thus heapify is linear. $\square$

Mental model. Most vertices are near the leaves, and near-leaf vertices cannot move far. That is why heapify is linear rather than $n \log n$.

29 Heap Sort and the impossibility of constant-time heap operations

29.1 Heap Sort using $\mathcal{O}(1)$ extra memory

Heap Sort sorts an array in place. For increasing order, it is convenient to use a max-heap. The largest element is repeatedly moved to the end of the active prefix.

```
HEAP-SORT(A):
    BUILD-MAX-HEAP(A)
    heapSize = n
    for end = n-1 downto 1:
        swap A[0], A[end]
        heapSize = heapSize - 1
        SIFT-DOWN-MAX(0)
```

Theorem 29.1 (Correctness of Heap Sort). *After $\text{HEAP-SORT}(A)$, the array is sorted in nondecreasing order.*

Proof. The invariant before each loop iteration is:

- $A[0 \dots \text{heapSize} - 1]$ is a max-heap;
- $A[\text{heapSize} \dots n - 1]$ is sorted;
- every element in the heap prefix is at most every element in the sorted suffix.

Initially the suffix is empty and the prefix is a max-heap. At an iteration, the maximum of the prefix is at $A[0]$. Swapping it with the last element of the prefix places it at the beginning of the suffix, exactly where the next largest element belongs. Reducing `heapSize` excludes it from the heap. `siftDown` restores the max-heap property in the remaining prefix. The invariant is preserved. When the heap size becomes 1, the whole array is sorted. $\square$

Proposition 29.1 (Complexity). *Heap Sort runs in $\mathcal{O}(n \log n)$ worst-case time and uses $\mathcal{O}(1)$ additional memory, not counting the input array.*

Proof. Heap construction costs $\mathcal{O}(n)$. Then there are $n - 1$ iterations, each performing one `siftDown` of cost $\mathcal{O}(\log n)$. The algorithm stores only a constant number of indices and temporary variables. $\square$

29.2 Why comparison heaps cannot have both operations in $\mathcal{O}(1)$

Theorem 29.2 (No comparison-based priority queue with constant insert and extractMin). *There is no comparison-based priority queue that supports both insert and extractMin in $\mathcal{O}(1)$ worst-case time for all sequences of operations.*

Proof. Assume such a structure exists. To sort n arbitrary comparable keys, insert all n keys and then call extractMin n times. The extracted sequence is sorted. Under the assumed bounds this sorting algorithm uses $\mathcal{O}(n)$ comparisons in the worst case.

This contradicts the decision-tree lower bound for comparison sorting, which states that every comparison sort needs $\Omega(n \log n)$ comparisons in the worst case. Therefore at least one of insert and extractMin cannot be $\mathcal{O}(1)$ in any comparison-based priority queue. $\square$

Common pitfalls. The impossibility statement is about comparison-based structures and exact sorted extraction. It does not rule out special integer priority queues under stronger assumptions on the key universe.

30 Deleting from a heap by pointer

30.1 Handles and indices

Deleting an arbitrary heap element efficiently requires knowing its current index. A pointer or handle usually stores a reference to the item and is updated whenever heap elements are swapped.

```
DELETE-BY-HANDLE(i):
    swap A[i], A[heapSize - 1]
    heapSize = heapSize - 1
    if i == heapSize:
        return
    if i > 0 and A[i] < A[parent(i)]:
        SIFT-UP(i)
    else:
        SIFT-DOWN(i)
```

Theorem 30.1 (Correctness). *DELETE-BY-HANDLE(i) removes the element at index i and restores the heap property.*

Proof. Swapping with the last element and decreasing the heap size removes the desired element while preserving the complete-tree shape. Only the element moved into position i can violate heap order; every other node keeps the same parent-child relations as before.

If the moved element is smaller than its parent, the only possible violation is upward, so siftUp restores the heap. Otherwise it is not smaller than its parent, so it can only be too large for its children; siftDown restores the heap. These are exactly the correctness preconditions of the two sift operations. $\square$

Proposition 30.1 (Complexity). *Deletion by pointer takes $\mathcal{O}(\log n)$ worst-case time and $\Theta(1)$ extra memory, assuming swaps update handles in $\Theta(1)$ time.*

Common pitfalls. If external handles are not updated after every swap, they become invalid. Then future decreaseKey or delete operations may modify the wrong element.

31 Deleting from a heap by value

31.1 Without an index map

If only a value x is given, the heap property does not allow binary search. The array is not globally sorted. In the worst case, finding an occurrence of x requires scanning the heap.

```
DELETE-BY-VALUE(x):
    i = -1
    for j = 0 to heapSize - 1:
        if A[j] == x:
            i = j
            break
    if i == -1: return false
    DELETE-BY-HANDLE(i)
    return true
```

Proposition 31.1. *Without additional indexing, deletion by value costs $\Theta(n)$ worst-case time.*

Proof. The scan costs $\mathcal{O}(n)$ and deletion after finding the element costs $\mathcal{O}(\log n)$, so the total is $\mathcal{O}(n)$. For a lower bound, an adversary may place the searched value at the last inspected position, or not place it at all. Since heap order does not determine a unique search direction for arbitrary values, every array entry may have to be inspected. Thus the worst-case time is $\Omega(n)$. $\square$

31.2 With an auxiliary map

If values are unique, maintain a dictionary

$\text{pos}[\text{value}] = \text{current heap index.}$

Then deletion by value becomes dictionary lookup plus deletion by handle. With hashing, this is expected $\mathcal{O}(\log n)$; with a balanced tree map, it is $\mathcal{O}(\log n)$.

For duplicate values, the map should store a set of indices for each value.

Common pitfalls. Duplicates make delete by value ambiguous. The operation must specify whether it deletes any occurrence, the first inserted occurrence, or a particular handle.

32 Binomial trees and binomial heaps

32.1 Binomial trees

Definition 32.1 (Binomial tree). The binomial tree B_0 is a single node. For $k \geq 1$, the binomial tree B_k is formed by linking two copies of B_{k-1} , making the root of one copy the leftmost child of the root of the other copy.

Proposition 32.1 (Basic properties). *The tree B_k has:*

- 2^k nodes;
- height k ;
- root degree k ;
- root children with roots of binomial trees $B_{k-1}, B_{k-2}, \dots, B_0$.

Proof. The number of nodes doubles at each step, so $|B_k| = 2^k$. The height increases by one when one B_{k-1} becomes a child of the other root, hence height k . The root gets one new child at each construction step, so its degree is k . The statement about children follows by induction from the recursive construction. $\square$

32.2 Binomial heaps

Definition 32.2 (Binomial heap). A binomial min-heap is a collection of heap-ordered binomial trees such that for every degree k there is at most one tree of degree k .

Because a heap with n nodes contains at most one tree of each degree, its tree degrees correspond to the binary representation of n .

Mental model. A binomial heap is a priority queue where tree degrees behave like binary digits. Merging two heaps is analogous to binary addition with carries.

32.3 Advantage over a binary heap

A binary heap supports insert, getMin, extractMin, and decreaseKey efficiently, but merging two arbitrary binary heaps is not naturally logarithmic; the standard way is to concatenate arrays and rebuild in $\mathcal{O}(n + m)$ time.

A binomial heap supports merge in $\mathcal{O}(\log(n + m))$ worst-case time. This is the main structural advantage.

33 Operations in a binomial heap

33.1 Linking two trees

Two binomial trees of the same degree can be linked in $\Theta(1)$ time. The larger root becomes a child of the smaller root, preserving the min-heap property.

```
LINK(T1, T2):
    // degrees are equal
    if T2.root.key < T1.root.key:
        swap T1, T2
    make T2.root a child of T1.root
    increase degree of T1.root
    return T1
```


33.2 Merge

To merge two binomial heaps, first merge their root lists by increasing degree. Then scan degrees from small to large and link trees of equal degree, carrying the result to the next degree.

Theorem 33.1 (Merge correctness). *The merge operation returns a valid binomial heap containing exactly all elements of the two input heaps.*

Proof. The root-list merge preserves all trees and only orders them by degree. Whenever two trees have the same degree, linking them produces one binomial tree of the next degree and preserves heap order because the smaller root becomes the parent. This is exactly the carry operation in binary addition. After all carries are resolved, no two roots have the same degree, every tree is heap-ordered, and no element has been added or lost. $\square$

33.3 Other operations

```

INSERT(x):
    create one-node heap Hx
    H = MERGE(H, Hx)

GET-MIN():
    scan all roots and return the minimum root

EXTRACT-MIN():
    find root r with minimum key
    remove r's tree from the root list
    reverse the list of r's children to obtain a binomial heap Hchildren
    H = MERGE(H, Hchildren)
    return r.key

DECREASE-KEY(node, newKey):
    require newKey <= node.key
    node.key = newKey
    while node has parent and node.key < parent.key:
        swap node.key with parent.key
        node = parent
  
```

Theorem 33.2 (Operation bounds). *For a binomial heap with n elements:*

Operation	Worst-case time
<i>merge</i>	$\mathcal{O}(\log n)$
<i>insert</i>	$\mathcal{O}(\log n)$
<i>getMin</i>	$\mathcal{O}(\log n)$ unless a min pointer is maintained
<i>extractMin</i>	$\mathcal{O}(\log n)$
<i>decreaseKey</i>	$\mathcal{O}(\log n)$

Proof. A heap with n nodes has at most $\lfloor \log_2 n \rfloor + 1$ roots, because it has at most one tree of each degree. Merge scans and links a constant amount per possible degree, hence costs $\mathcal{O}(\log n)$. Insert is merge with a one-node heap. *getMin* scans all roots. *extractMin* scans roots, removes one root, reverses at most $\mathcal{O}(\log n)$ children, and merges. *decreaseKey* moves an element upward along a path of height at most $\mathcal{O}(\log n)$. $\square$

Remark 33.1 (Swapping keys versus nodes). For `decreaseKey`, swapping keys is simple. If external handles must always refer to the same logical item, swap whole item records or maintain handle metadata carefully.

34 Amortized analysis and accounting time

34.1 Definition

Worst-case analysis bounds every operation separately. Amortized analysis bounds the total cost of a sequence, even if some individual operations are expensive.

Definition 34.1 (Amortized cost). Let c_i be the actual cost of the i -th operation in a sequence. Numbers a_i are valid amortized costs if for every prefix of length m ,

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m a_i + C,$$

where C is a constant independent of m . Often $C = 0$.

Interpretation. An operation may be charged more than its actual cost. The surplus pays for future expensive operations. The condition must hold for every prefix, not only for the whole sequence chosen after the fact.

34.2 Aggregate method

The simplest amortized proof is aggregate analysis: prove directly that any sequence of m operations costs $\mathcal{O}(m)$ total. Then the amortized cost per operation is $\mathcal{O}(1)$.

Example 34.1 (Two-stack queue). In a two-stack queue, each element is pushed into in, moved once to out, and popped once. Hence m queue operations perform $\mathcal{O}(m)$ stack operations. The amortized cost is $\mathcal{O}(1)$.

34.3 Potential method

Another common method assigns a nonnegative potential $\Phi(D)$ to the current data structure state D . Define

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Then

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m c_i + \Phi(D_m) - \Phi(D_0).$$

If $\Phi(D_m) \geq \Phi(D_0)$, then total actual cost is at most total amortized cost.

Common pitfalls. Amortized $\mathcal{O}(1)$ is not the same as worst-case $\mathcal{O}(1)$. One operation may still take $\Theta(n)$ time, but such expensive operations cannot occur too often.

35 The coin method: accounting analysis

35.1 Coin method

The coin method assigns an artificial charge to operations. Some coins pay for the current operation; extra coins are stored on objects or inside the data structure to pay for future operations.

Definition 35.1 (Accounting proof). An accounting proof is valid if the number of stored coins is never negative and every actual elementary operation is paid for by a coin. Then the total actual cost is at most the total charged amortized cost.

35.2 Example: queue on two stacks

Charge 3 coins for each `enqueue(x)`:

- 1 coin pays for pushing x into `in`;
- 1 coin is stored on x to pay for its future move from `in` to `out`;
- 1 coin is stored on x to pay for its future pop from `out`.

Then `dequeue()` either pops directly from `out`, paid by the element's stored pop coin, or first moves elements from `in` to `out`, each move paid by the corresponding move coin.

Theorem 35.1 (Accounting bound). *The two-stack queue has $\mathcal{O}(1)$ amortized cost per operation under the coin method.*

Proof. Every element receives enough coins at insertion to pay for all future stack operations involving that element: one push into `in`, one transfer push/pop pair abstracted as a constant-cost move, and one final pop from `out`. No operation ever requires coins that have not already been stored. Therefore the total actual number of elementary stack operations over any prefix is bounded by a constant times the number of queue operations in that prefix. $\square$

Mental model. Coins are not real memory. They are a proof device. Their purpose is to show that expensive operations have already been prepaid by earlier cheap operations.

36 Vector data structure: dynamic array

36.1 Array implementation

A vector, or dynamic array, stores elements in a contiguous array with two parameters:

`size` = number of stored elements, `capacity` = allocated array length.

Elements occupy positions $0, \dots, \text{size} - 1$.

```
PUSH-BACK(x):
  if size == capacity:
    newCapacity = max(1, 2 * capacity)
    allocate B[0 .. newCapacity-1]
```

```

    copy A[0 .. size-1] into B
    A = B
    capacity = newCapacity
    A[size] = x
    size = size + 1

```

36.2 Operation costs

Operation	Worst-case time	Amortized time
Read/write by index	$\Theta(1)$	$\Theta(1)$
pushBack	$\Theta(n)$ during resize	$\Theta(1)$
popBack without shrinking	$\Theta(1)$	$\Theta(1)$
Insert/delete in middle	$\Theta(n)$	$\Theta(n)$

Theorem 36.1 (Amortized constant push-back). *If the capacity doubles whenever the array is full, then pushBack has $\mathcal{O}(1)$ amortized time.*

Proof. Consider a sequence of m pushes starting from an empty vector. Resizes happen when capacities are $1, 2, 4, 8, \dots$. The total number of copied elements is bounded by

$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 m \rfloor} < 2m.$$

In addition, each push writes its new element once. Thus the total work is $\mathcal{O}(m)$, so the amortized time per push is $\mathcal{O}(1)$. $\square$

36.3 Shrinking

If capacity is halved immediately whenever $\text{size} < \text{capacity}$, alternating push and pop can cause repeated resizing. A standard safe rule is to halve capacity only when

$$\text{size} \leq \frac{\text{capacity}}{4}.$$

This hysteresis prevents resize thrashing.

Common pitfalls. Dynamic arrays are fast for random access and appending. They are not fast for inserting at the beginning, because all existing elements must be shifted.

37 Sparse Table

37.1 Model problem: static range minimum query

Given a static array $A[0 \dots n-1]$, answer queries

$$\text{RMQ}(l, r) = \min\{A[l], A[l+1], \dots, A[r]\}$$

in $\mathcal{O}(1)$ time after preprocessing.

Sparse Table works for static arrays and idempotent associative operations, such as minimum, maximum, gcd. The key identity is

$$\min(X \cup X) = \min(X),$$

so overlapping intervals are allowed.

37.2 Construction

Let

$$st[k][i] = \min\{A[i], A[i+1], \dots, A[i+2^k-1]\}.$$

Then

$$st[0][i] = A[i],$$

and for $k > 0$,

$$st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}]).$$

```
BUILD-SPARSE-TABLE(A):
  for i = 0 to n-1:
    st[0][i] = A[i]
  for k = 1 while 2^k <= n:
    for i = 0 while i + 2^k <= n:
      st[k][i] = min(st[k-1][i], st[k-1][i + 2^(k-1)])
```

37.3 Query

For interval length $L = r - l + 1$, let $k = \lfloor \log_2 L \rfloor$. Then two intervals of length 2^k cover $[l, r]$:

$$[l, l + 2^k - 1], \quad [r - 2^k + 1, r].$$

They may overlap, but minimum is idempotent, so this is harmless.

```
RMQ(l, r):
  L = r - l + 1
  k = floorLog2[L]
  return min(st[k][l], st[k][r - 2^k + 1])
```

Theorem 37.1 (Sparse Table correctness). *The query algorithm returns the minimum on $A[l \dots r]$.*

Proof. By construction, $st[k][l]$ is the minimum on the left block of length 2^k , and $st[k][r-2^k+1]$ is the minimum on the right block of length 2^k . Since $2^k \leq L < 2^{k+1}$, these two blocks cover the whole interval $[l, r]$. The minimum over the union is the minimum of the two block minima. Overlap does not change the minimum. $\square$

Proposition 37.1 (Complexity). *Sparse Table preprocessing costs $\mathcal{O}(n \log n)$ time and memory. Each RMQ query costs $\mathcal{O}(1)$ time.*

Common pitfalls. For sums, the overlapping-block trick is invalid because sum is not idempotent. Sparse Table can answer range sums with disjoint power-of-two decomposition in $\mathcal{O}(\log n)$, but not in this $\mathcal{O}(1)$ form.

38 Segment tree: model problem and query time

38.1 Model problem

A segment tree stores information about intervals. A standard model problem is dynamic range sum:

- `set(pos, value)`: update one array element;
- `sum(l, r)`: return $A[l] + \dots + A[r]$.

Each vertex represents a segment $[L, R]$. The root represents $[0, n - 1]$. If $L < R$, split at $M = \lfloor (L + R)/2 \rfloor$ into $[L, M]$ and $[M + 1, R]$.

Definition 38.1 (Segment value). For the range-sum tree, each vertex v stores

$$tree[v] = \sum_{i=L_v}^{R_v} A[i].$$

38.2 Build and point update

```

BUILD(v, L, R):
    if L == R:
        tree[v] = A[L]
    else:
        M = (L + R) / 2
        BUILD(2v, L, M)
        BUILD(2v+1, M+1, R)
        tree[v] = tree[2v] + tree[2v+1]

SET(v, L, R, pos, value):
    if L == R:
        tree[v] = value
    else:
        M = (L + R) / 2
        if pos <= M: SET(2v, L, M, pos, value)
        else: SET(2v+1, M+1, R, pos, value)
        tree[v] = tree[2v] + tree[2v+1]

```

38.3 Range query

```

SUM(v, L, R, ql, qr):
    if qr < L or R < ql:
        return 0
    if ql <= L and R <= qr:
        return tree[v]
    M = (L + R) / 2
    return SUM(2v, L, M, ql, qr)
        + SUM(2v+1, M+1, R, ql, qr)

```

Theorem 38.1 (Correctness). $SUM(1, 0, n-1, l, r)$ returns $\sum_{i=l}^r A[i]$.

Proof. For any recursive call on node segment $[L, R]$, there are three cases. If $[L, R]$ is disjoint from $[l, r]$, its contribution is 0. If $[L, R]$ is fully contained in $[l, r]$, the stored value is exactly the sum over that segment. Otherwise the query intersects both or part of the children, and the desired contribution is the sum of the desired contributions from the left and right child segments. By induction on segment length, the recursive formula returns exactly the sum over $[L, R] \cap [l, r]$. For the root this is the desired range sum. $\square$

Theorem 38.2 (Query time). A segment-tree range query visits $\mathcal{O}(\log n)$ relevant boundary nodes and $\mathcal{O}(\log n)$ fully covered nodes; hence it runs in $\mathcal{O}(\log n)$ time.

Proof. At each tree level, the query interval $[l, r]$ can partially intersect at most two nodes: one containing the left boundary and one containing the right boundary. All other intersected nodes at that level are fully contained in the query and do not recurse further. Since the tree has height $\mathcal{O}(\log n)$, the number of recursive calls is $\mathcal{O}(\log n)$. $\square$

Common pitfalls. The neutral value depends on the operation: for sums it is 0, for minima it is $+\infty$, for maxima it is $-\infty$.

39 Segment tree binary descent: finding a segment with sum at least x

39.1 Prefix lower-bound query

Assume $A[i] \geq 0$ for all i . Store segment sums. Given x , find the smallest index p such that

$$A[0] + A[1] + \dots + A[p] \geq x.$$

If the total sum is less than x , no such index exists.

```

LOWER-BOUND-PREFIX(x):
    if tree[1] < x:
        return NOT_FOUND
    v = 1; L = 0; R = n-1
    while L != R:
        M = (L + R) / 2
        if tree[2v] >= x:
            v = 2v
            R = M
        else:
            x = x - tree[2v]
            v = 2v + 1
            L = M + 1
    return L

```

Theorem 39.1 (Correctness of binary descent). *If all array values are nonnegative, LOWER-BOUND-PREFIX(x) returns the smallest index whose prefix sum is at least x .*

Proof. At a node segment $[L, R]$, maintain the invariant: the desired answer lies inside $[L, R]$, and x is the remaining amount that must be reached within this segment after subtracting everything strictly before L .

Let the left child be $[L, M]$. If its sum is at least x , then the first position reaching the required amount is inside the left child, because all values are nonnegative and prefixes grow monotonically. Otherwise the whole left child is insufficient, so the answer must lie in the right child, and the remaining required amount becomes $x - \text{tree}[2v]$. This preserves the invariant. At a leaf, the unique position is the answer. $\square$

39.2 Range version

To find the smallest $r \geq l$ such that

$$A[l] + A[l+1] + \dots + A[r] \geq x,$$

one can descend through the segment tree while ignoring nodes completely before l and carrying an accumulator. The same monotonicity assumption $A[i] \geq 0$ is essential.

Common pitfalls. If negative numbers are allowed, prefix sums are not monotone. Then the binary descent based only on segment sums is invalid.

40 Segment tree with lazy propagation: range assignment and push

40.1 Problem

Support operations:

- `assign(l, r, x)`: set every $A[i]$ with $l \leq i \leq r$ to x ;
- `sum(l, r)`: return the sum on a range.

A naive assignment may touch $\Theta(n)$ leaves. Lazy propagation stores delayed operations at internal vertices.

40.2 Lazy assignment tag

For each vertex v representing $[L, R]$, store:

- `tree[v]`: sum over $[L, R]$ after all logically applied operations;
- `hasLazy[v]`: whether this whole segment has a pending assignment;
- `lazy[v]`: the value to assign to all children when needed.

```

APPLY(v, L, R, x):
    tree[v] = (R - L + 1) * x
    hasLazy[v] = true
    lazy[v] = x

PUSH(v, L, R):
    if not hasLazy[v] or L == R:
        return
    M = (L + R) / 2
    APPLY(2v, L, M, lazy[v])
    APPLY(2v+1, M+1, R, lazy[v])
    hasLazy[v] = false

```

40.3 Range assignment

```

ASSIGN(v, L, R, ql, qr, x):
    if qr < L or R < ql:
        return
    if ql <= L and R <= qr:
        APPLY(v, L, R, x)
        return
    PUSH(v, L, R)
    M = (L + R) / 2
    ASSIGN(2v, L, M, ql, qr, x)
    ASSIGN(2v+1, M+1, R, ql, qr, x)
    tree[v] = tree[2v] + tree[2v+1]

```


Queries are the ordinary segment-tree range queries, except that PUSH is called before descending to children.

Theorem 40.1 (Correctness of lazy assignment). *After any sequence of assignments, every $tree[v]$ equals the sum of its segment in the logical array, and pending tags correctly represent assignments not yet pushed to descendants.*

Proof. APPLY(v, L, R, x) is correct because assigning all $R - L + 1$ elements of the segment to x makes the segment sum equal to $(R - L + 1)x$. The lazy tag records that descendants should also be considered assigned to x when they are later accessed.

PUSH sends the pending assignment to both children. Since assignment overwrites older values and older pending assignments, replacing the children tags by x is correct. After pushing, the parent no longer needs a pending tag.

For ASSIGN, disjoint nodes are unchanged, fully covered nodes are handled correctly by APPLY, and partially covered nodes first push pending information so that children are consistent. The recursive calls correctly update child segments, and recomputing the parent as the sum of the two children restores the invariant. $\square$

Proposition 40.1 (Complexity). *Range assignment and range-sum query both run in $\mathcal{O}(\log n)$ time.*

Proof. Lazy propagation ensures that a fully covered segment is processed in $\Theta(1)$ time without descending to all leaves. As in the ordinary segment tree, only $\mathcal{O}(\log n)$ boundary nodes are expanded. Each expansion performs $\mathcal{O}(1)$ push work. $\square$

Common pitfalls. For range assignment, lazy tags overwrite older assignment tags. This differs from range addition, where tags are usually accumulated by addition.

41 Counting values at most x on a range: Fractional Cascading

41.1 Model problem

Given a static array $A[0 \dots n - 1]$, answer queries

$$\#\{i \in [l, r] : A[i] \leq x\}.$$

A merge-sort tree solves this in $\mathcal{O}(\log^2 n)$ time: decompose $[l, r]$ into $\mathcal{O}(\log n)$ segment-tree nodes and binary-search x in each node's sorted list.

Fractional Cascading reduces the repeated binary searches to one binary search plus $\mathcal{O}(1)$ work per visited node.

41.2 Merge-sort tree

At each segment-tree node v , store a sorted list

$$list[v] = \text{sorted values of } A[L_v \dots R_v].$$

Then for a fully covered node v , the number of values at most x is

$$\text{upper_bound}(list[v], x),$$

where the result is the number of entries $\leq x$.

41.3 Fractional Cascading pointers

For every internal node v with children u and w , precompute arrays

$$\text{toLeft}_v[p] = \#\{y \in \text{list}[u] : y \leq \text{list}[v][p-1]\},$$

$$\text{toRight}_v[p] = \#\{y \in \text{list}[w] : y \leq \text{list}[v][p-1]\},$$

for positions $p = 0, 1, \dots, |\text{list}[v]|$. Equivalently, if $p = \text{upper_bound}(\text{list}[v], x)$, then $\text{toLeft}[p]$ and $\text{toRight}[p]$ give the corresponding upper-bound positions in the child lists.

In implementation, these arrays are constructed during the merge of two child sorted lists.

41.4 Query idea

First compute

$$p_0 = \text{upper_bound}(\text{list}[\text{root}], x)$$

by one binary search. During the recursive range query, whenever the recursion moves from a node to a child, translate the current position p to the child position using the precomputed pointer. Thus no new binary search is needed.

```

COUNT(v, L, R, ql, qr, p):
    // p = upper_bound(list[v], x)
    if qr < L or R < ql:
        return 0
    if ql <= L and R <= qr:
        return p
    M = (L + R) / 2
    pLeft = toLeft[v][p]
    pRight = toRight[v][p]
    return COUNT(2v, L, M, ql, qr, pLeft)
        + COUNT(2v+1, M+1, R, ql, qr, pRight)

```

Theorem 41.1 (Correctness). *The fractional-cascading merge-sort tree returns the number of indices $i \in [l, r]$ such that $A[i] \leq x$.*

Proof. For every visited node v , maintain the invariant that p equals the number of elements in $\text{list}[v]$ that are at most x . At the root this is true by the initial binary search. If the recursion moves to a child, the precomputed pointer maps the parent's upper-bound position to the child's upper-bound position, so the invariant is preserved.

A disjoint node contributes 0. A fully covered node contributes exactly p , the number of values at most x in its segment. A partially covered node splits the query between its children, whose segments partition the parent segment. Summing the two recursive answers gives the correct count. $\square$

Proposition 41.1 (Complexity). *The data structure uses $\mathcal{O}(n \log n)$ memory and can be built in $\mathcal{O}(n \log n)$ time. Each query takes $\mathcal{O}(\log n)$ time: one binary search at the root and $\mathcal{O}(1)$ work per visited segment-tree node.*

Common pitfalls. Use `upper__ESC_UND__bound` for counting values $\leq x$ and `lower__ESC_UND__bound` for counting values $< x$. Mixing these two is the most common off-by-one error in this topic.

Exercise 41.1. Show how to answer the number of values in $[l, r]$ lying in an interval $[a, b]$ using two queries of the above form.

Compact complexity table

Structure / operation	Time	Notes
Min-stack	$\mathcal{O}(1)$ worst-case per operation	store prefix minimum with each value
Linked-list queue	$\mathcal{O}(1)$ worst-case	requires head and tail
Two-stack queue	$\mathcal{O}(1)$ amortized	one expensive transfer may occur
Queue minimum	$\mathcal{O}(1)$ amortized	two min-stacks
Binary heap <code>getMin</code>	$\mathcal{O}(1)$	root
Binary heap <code>insert</code>	$\mathcal{O}(\log n)$	<code>siftUp</code>
Binary heap <code>extractMin</code>	$\mathcal{O}(\log n)$	<code>siftDown</code>
Binary heap <code>decreaseKey</code>	$\mathcal{O}(\log n)$	needs handle or index
Heapify	$\mathcal{O}(n)$	bottom-up construction
Heap Sort	$\mathcal{O}(n \log n)$, $\mathcal{O}(1)$ extra memory	not stable by default
Delete heap element by pointer	$\mathcal{O}(\log n)$	update handles on swaps
Delete heap element by value	$\Theta(n)$ without map	heap is not sorted
Binomial heap merge	$\mathcal{O}(\log n)$	main advantage over binary heap
Dynamic array <code>pushBack</code>	$\mathcal{O}(1)$ amortized	doubling capacity
Sparse Table RMQ	build $\mathcal{O}(n \log n)$, query $\mathcal{O}(1)$	static, idempotent operations
Segment tree range query	$\mathcal{O}(\log n)$	dynamic updates
Segment tree lazy assignment	$\mathcal{O}(\log n)$	delayed tags
Fractional Cascading count	build $\mathcal{O}(n \log n)$, query $\mathcal{O}(\log n)$	static range counting

42 Dynamic segment tree

42.1 Problem and motivation

A usual segment tree allocates all nodes for an array of known length n . A *dynamic segment tree*, also called an *implicit segment tree*, is used when the coordinate universe is huge, for example $[0, 10^9)$, but only a small number of points or intervals is ever touched.

The model problem is:

- initially all values are zero on a universe $[L, R)$;
- support point updates $a[p] \leftarrow a[p] + \Delta$;
- support range-sum queries $\sum_{i \in [q_l, q_r)} a[i]$.

The key requirement is that memory must depend on the number of performed updates, not on $R - L$.

Mental model. A dynamic segment tree is the same recursive interval decomposition as an ordinary segment tree, except nodes are created only when the algorithm

actually needs them. A missing child represents an all-zero subtree.

42.2 Node representation

A node stores an interval $[l, r)$ implicitly through recursion and contains

sum, left, right.

The child intervals are

$$[l, m), \quad [m, r), \quad m = \left\lfloor \frac{l + r}{2} \right\rfloor.$$

If a child pointer is null, the corresponding subtree is treated as having aggregate value 0.

Invariant. For every allocated node representing $[l, r)$, `node.sum` equals the sum of all currently stored values in $[l, r)$. Null children represent zero intervals and therefore contribute zero to this invariant.

42.3 Point update and range query

```
POINT-ADD(v, l, r, p, delta):
    if v == null:
        v = new Node(sum = 0, left = null, right = null)
    if r - l == 1:
        v.sum += delta
        return v

    m = floor((l + r) / 2)
    if p < m:
        v.left = POINT-ADD(v.left, l, m, p, delta)
    else:
        v.right = POINT-ADD(v.right, m, r, p, delta)

    v.sum = SUM(v.left) + SUM(v.right)
    return v

RANGE-SUM(v, l, r, ql, qr):
    if v == null or qr <= l or r <= ql:
        return 0
    if ql <= l and r <= qr:
        return v.sum

    m = floor((l + r) / 2)
    return RANGE-SUM(v.left, l, m, ql, qr)
        + RANGE-SUM(v.right, m, r, ql, qr)

def SUM(v):
    if v == null: return 0
    else: return v.sum
```

Proposition 42.1 (Correctness of dynamic point update). *After a call to `POINT-ADD` with parameters $(\text{root}, L, R, p, \Delta)$, the represented array is changed only at position p , and the invariant holds for every allocated node.*

Proof. The proof is by induction on the interval length $r-l$. If $r-l = 1$, then the interval contains exactly one coordinate. The algorithm adds Δ to the stored sum, so the leaf invariant is correct.

For $r-l > 1$, the point p belongs to exactly one of the two child intervals. By the induction hypothesis, the recursive call correctly updates that child and preserves its invariant. The other child is unchanged. After the recursive call, the algorithm recomputes

$$v.\text{sum} = \text{SUM}(v.\text{left}) + \text{SUM}(v.\text{right}),$$

which is exactly the sum over $[l, r)$ because $[l, r)$ is the disjoint union of its children. Thus the invariant is restored at v and remains true everywhere else. $\square$

Proposition 42.2 (Correctness of range query). *RANGE-SUM($root, L, R, ql, qr$) returns $\sum_{i \in [ql, qr)} a[i]$.*

Proof. The recursive cases follow the standard segment-tree trichotomy. If the node interval is disjoint from the query, the contribution is 0. If it is fully contained in the query, the invariant gives the correct contribution immediately. Otherwise the interval is split into its two children; the intersection with the query decomposes into the disjoint union of the two child intersections. By induction on interval length, the two recursive answers are correct, and their sum is the desired answer. $\square$

Theorem 42.1 (Complexity). *If the universe length is $U = R - L$, then a point update costs $\mathcal{O}(\log U)$ time and creates at most $\mathcal{O}(\log U)$ nodes. A range query costs $\mathcal{O}(\log U)$ time in the standard one-dimensional segment-tree sense: it visits at most $\mathcal{O}(\log U)$ canonical boundary nodes plus fully covered nodes needed for the answer.*

Remark 42.1. For a general range query, the number of visited nodes is $\mathcal{O}(\log U + k)$ where k is the number of reported canonical fully covered nodes. For aggregate queries such as sums, the canonical decomposition has size $\mathcal{O}(\log U)$.

Common pitfalls. Do not allocate children during a pure query. A null child means an all-zero interval. Allocating during queries destroys the memory advantage.

Exercise 42.1. Modify the dynamic segment tree to support point assignment $a[p] \leftarrow x$ instead of addition. State the invariant and prove correctness.

43 Coordinate compression for segment trees

43.1 The idea

Coordinate compression replaces large actual coordinates by their ranks in a sorted list of relevant coordinates. It is used when values lie in a huge domain but only m distinct coordinates matter.

Suppose all coordinates that will ever appear in updates or queries are known offline. Let

$$X = \{x_0 < x_1 < \dots < x_{m-1}\}$$

be the sorted list of distinct relevant coordinates. The compressed coordinate of x_i is i .

Mental model. Compression does not preserve distances. It preserves order. Therefore it is correct for data structures that only need order or point identities, but one must be careful with interval lengths.

43.2 Point-based compression

For point updates and range queries over values indexed by coordinates, define

$$\text{comp}(x) = \text{the index of } x \text{ in } X.$$

A range $[l, r]$ over original coordinates is mapped to index interval

$$[\text{lower_bound}(X, l), \text{upper_bound}(X, r)).$$

For half-open ranges $[l, r)$ use

$$[\text{lower_bound}(X, l), \text{lower_bound}(X, r)).$$

Proposition 43.1 (Order preservation). *For any relevant coordinates $x, y \in X$,*

$$x < y \iff \text{comp}(x) < \text{comp}(y).$$

Proof. The list X is sorted strictly increasingly. The index function is precisely the order position in that list. $\square$

43.3 Segment compression for interval lengths

Sometimes the data structure stores information about elementary intervals, not only points. Then the relevant elementary segments are

$$[X_0, X_1), [X_1, X_2), \dots, [X_{m-2}, X_{m-1}).$$

If a segment tree stores total covered length, the leaf corresponding to i must have length $X_{i+1} - X_i$, not length 1.

Example 43.1. If coordinates are 10, 100, 101, then compressed indices are 0, 1, 2, but the interval from 10 to 100 has length 90, while the interval from 100 to 101 has length 1. Treating both compressed gaps as length 1 is wrong for geometric union-length problems.

43.4 Complexity

Building the coordinate list costs $\mathcal{O}(m \log m)$ time for sorting and deduplication. A segment tree over compressed coordinates has size $\mathcal{O}(m)$ and operation time $\mathcal{O}(\log m)$.

Common pitfalls. The most common error is forgetting which kind of object is compressed: points or gaps between points. Range-counting problems usually compress points. Covered-length problems usually compress gaps.

Exercise 43.1. Given offline operations of the form $\text{add}(x, \text{delta})$ and $\text{sum}(l, r)$ over integer coordinates up to 10^{18} , describe exactly which coordinates must be included in the compressed list.

44 Persistent segment tree

44.1 Persistence

A data structure is *persistent* if old versions remain available after updates. A persistent segment tree usually uses *path copying*: when updating a leaf, allocate new nodes only along the root-to-leaf path, and reuse all untouched subtrees.

Definition 44.1 (Partial persistence). A data structure is partially persistent if every version can be queried, but updates are applied only to the newest version or create a new version from a specified old version.

Mental model. A persistent segment tree is a history of roots. The roots differ, but most of their subtrees are shared. Each update creates a new spine of nodes, while the rest of the tree is immutable and reused.

44.2 Persistent point update

Assume the tree stores sums on a fixed domain $[0, n)$.

```
PERSISTENT-ADD(v, l, r, p, delta):
    u = copy of v
    if r - l == 1:
        u.sum = v.sum + delta
        return u

    m = floor((l + r) / 2)
    if p < m:
        u.left = PERSISTENT-ADD(v.left, l, m, p, delta)
        u.right = v.right
    else:
        u.left = v.left
        u.right = PERSISTENT-ADD(v.right, m, r, p, delta)

    u.sum = u.left.sum + u.right.sum
    return u
```

The new root returned by the update represents the new version. The old root still points to the old nodes and is unchanged.

Invariant. Every root represents one immutable version of the array. For every node reachable from a root, its stored sum is the sum of the corresponding interval in that root's version.

Proposition 44.1 (Correctness of path copying). *The persistent update creates a new version equal to the old version except for the updated position p , while every old version remains unchanged.*

Proof. The algorithm never mutates an existing node; it only reads old nodes and allocates new ones. Therefore all old roots still reach exactly the same nodes as before, so old versions are unchanged.

For the new root, correctness is proved by induction on interval length. At a leaf, the sum is changed exactly by Δ . At an internal node, exactly one child interval contains

p and is recursively updated; the other child is reused because its array values are unchanged. Recomputing the parent sum from the two child sums restores the invariant. Thus the new root represents precisely the updated array. $\square$

Theorem 44.1 (Complexity). *Each point update creates $\mathcal{O}(\log n)$ new nodes and costs $\mathcal{O}(\log n)$ time. Each range query on any version costs $\mathcal{O}(\log n)$ time. If q updates are performed, total memory is $\mathcal{O}(n + q \log n)$ if the initial tree is built explicitly, or $\mathcal{O}(q \log n)$ if zero nodes are created lazily.*

Common pitfalls. Persistence fails if a reused child is later modified in place. Once nodes are shared between versions, they must be treated as immutable.

Exercise 44.1. Write persistent point assignment $a[p] \leftarrow x$ and explain how it differs from persistent point addition.

45 Counting numbers not exceeding x on a subarray: persistent segment tree solution

45.1 Model problem

Given a static array

$$a_1, a_2, \dots, a_n,$$

answer queries

$$\#\{i \in [l, r] : a_i \leq x\}.$$

The standard persistent solution builds one version per prefix.

45.2 Prefix versions over values

First compress the values of the array:

$$b_i = \text{rank}(a_i) \in \{0, \dots, m-1\}.$$

Let $\text{root}[0]$ be the empty frequency tree. For $i = 1, \dots, n$, define

$$\text{root}[i] = \text{root}[i-1] + \text{one occurrence of } b_i.$$

Thus $\text{root}[i]$ stores the frequency multiset of values in the prefix $a_1, \dots, a_i$.

```
BUILD-PREFIX-ROOTS(a[1..n]):
  X = sorted distinct values of a
  root[0] = zero tree over [0, |X|)
  for i = 1..n:
    p = lower_bound(X, a[i])
    root[i] = PERSISTENT-ADD(root[i-1], 0, |X|, p, +1)
```

For query (l, r, x) , let

$$t = \text{upper_bound}(X, x).$$

Then the answer is

$$\text{sum}(\text{root}[r], [0, t)) - \text{sum}(\text{root}[l-1], [0, t)).$$

Theorem 45.1 (Correctness). *The above algorithm returns the number of indices $i \in [l, r]$ such that $a_i \leq x$.*

Proof. By construction, `root[r]` stores frequencies of all values in positions $1, \dots, r$, and `root[l-1]` stores frequencies of all values in positions $1, \dots, l-1$. Therefore their difference stores frequencies of exactly positions $l, \dots, r$.

The compressed values less than $t = \text{upper_bound}(X, x)$ are precisely those original values $\leq x$. Summing the frequency difference over $[0, t)$ counts exactly the desired indices. $\square$

Theorem 45.2 (Complexity). *Preprocessing costs $\mathcal{O}(n \log n)$ time for sorting and compression. The persistent updates add $\mathcal{O}(n \log m)$ time and memory, where m is the number of distinct values. Each query costs $\mathcal{O}(\log m)$ time.*

Common pitfalls. Use `upper__ESC_UND__bound` for the condition $\leq x$. Using `lower__ESC_UND__bound` would count values $< x$.

Exercise 45.1. Modify the query to count values in $[A, B]$ on a subarray $[l, r]$.

46 Range k-th order statistic via persistent segment trees and parallel descent

46.1 Problem

Given a static array $a_1, \dots, a_n$, answer queries:

find the k -th smallest value among $a_l, a_{l+1}, \dots, a_r$.

Assume $1 \leq k \leq r - l + 1$.

The same prefix persistent trees from the previous section are used. The query works by descending simultaneously in two roots: `root[r]` and `root[l-1]`.

Mental model. The difference of two prefix trees represents the multiset on the query range. To find the k -th value, ask: how many query elements go to the left half? If at least k , the answer is in the left half; otherwise skip those elements and go right with a smaller rank.

46.2 Parallel descent

Let $R = \text{root}[r]$ and $L = \text{root}[l-1]$. At a node interval $[lo, hi)$, define

$$c_{\text{left}} = R.\text{left.sum} - L.\text{left.sum}.$$

This is the number of query-range elements whose compressed values lie in the left child interval.

```
KTH(L, R, lo, hi, k):
    if hi - lo == 1:
        return lo

    mid = floor((lo + hi) / 2)
    countLeft = R.left.sum - L.left.sum
```

```

if k <= countLeft:
    return KTH(L.left, R.left, lo, mid, k)
else:
    return KTH(L.right, R.right, mid, hi, k - countLeft)

```

The returned compressed index is converted back to the original value through the sorted value list X .

Theorem 46.1 (Correctness). $KTH(root[l-1], root[r], 0, m, k)$ returns the compressed value of the k -th smallest element of $a_l, \dots, a_r$.

Proof. For any node interval I , the difference between the two persistent trees gives the frequency multiset of query elements whose compressed values lie in I . This follows from the prefix-difference argument.

At an internal node, the interval is split into left and right halves. All values in the left half are smaller than all values in the right half. If $c_{left} \geq k$, then the k -th smallest value among the elements in I must lie in the left half, and its rank inside that half remains k . If $c_{left} < k$, then all left-half elements precede the answer, so the answer lies in the right half and becomes the $(k - c_{left})$ -th smallest element there. This preserves the query invariant.

The recursion stops at a leaf, which represents a single compressed value. The invariant implies that this leaf is exactly the desired order statistic. $\square$

Theorem 46.2 (Complexity). Each query costs $\mathcal{O}(\log m)$ time and $\mathcal{O}(1)$ additional memory beyond recursion stack, where m is the number of distinct values.

Common pitfalls. The two roots must correspond to prefixes r and $l - 1$. Using l instead of $l - 1$ removes position l from the query interval.

Exercise 46.1. Show how to answer the median query on a subarray using the same method.

47 Fenwick tree: classical problem, update and getSum

47.1 Model problem

A Fenwick tree, or Binary Indexed Tree, stores an array $a_1, \dots, a_n$ and supports:

- `add(i, delta)`: $a_i \leftarrow a_i + \Delta$;
- `prefixSum(i)`: return $a_1 + \dots + a_i$.

Both operations take $\mathcal{O}(\log n)$ time.

47.2 The lowbit decomposition

For $i > 0$, define

$$\text{lowbit}(i) = i \& (-i),$$

which is the largest power of two dividing i .

Fenwick cell $t[i]$ stores the sum over the interval

$$(i - \text{lowbit}(i), i]$$

in one-based indexing.

Invariant. For every i ,

$$t[i] = \sum_{j=i-\text{lowbit}(i)+1}^i a_j.$$

47.3 Operations

```

ADD(i, delta):
    while i <= n:
        t[i] += delta
        i += lowbit(i)

PREFIX-SUM(i):
    ans = 0
    while i > 0:
        ans += t[i]
        i -= lowbit(i)
    return ans

RANGE-SUM(l, r):
    return PREFIX-SUM(r) - PREFIX-SUM(l - 1)

```

Proposition 47.1 (Correctness of prefix sum). *PREFIX-SUM(i) returns $\sum_{j=1}^i a_j$.*

Proof. At each step, the algorithm adds $t[i]$, which by the invariant is the sum over

$$[i - \text{lowbit}(i) + 1, i].$$

Then it replaces i by $i - \text{lowbit}(i)$, so the next interval ends exactly where the previous one began minus one. Thus the algorithm partitions $[1, i]$ into disjoint Fenwick intervals and sums all of them. $\square$

Proposition 47.2 (Correctness of update). *After $\text{ADD}(i, \text{delta})$, the Fenwick invariant holds for the array with a_i increased by Δ .*

Proof. Cell $t[p]$ must change exactly when its stored interval contains i , that is when

$$p - \text{lowbit}(p) + 1 \leq i \leq p.$$

The sequence

$$i, i + \text{lowbit}(i), i + \text{lowbit}(i + \text{lowbit}(i)), \dots$$

visits exactly those Fenwick cells whose intervals contain i . Therefore adding Δ to all visited cells updates exactly the affected stored sums and preserves every other cell. $\square$

Theorem 47.1 (Complexity). *Both operations perform at most $\lfloor \log_2 n \rfloor + 1$ iterations, so their time complexity is $\mathcal{O}(\log n)$ and memory is $\mathcal{O}(n)$.*

Common pitfalls. Fenwick trees are easiest with one-based indexing. In zero-based indexing, the formulas change; mixing the two conventions is a common source of infinite loops.

Exercise 47.1. Prove that PREFIX-SUM visits at most one Fenwick interval of each power-of-two length.

48 Higher-dimensional Fenwick trees

48.1 Two-dimensional model problem

For a matrix $a[x][y]$ of size $n \times m$, support:

- point update $a[x][y] \leftarrow a[x][y] + \Delta$;
- rectangle prefix sum $\sum_{1 \leq i \leq x, 1 \leq j \leq y} a[i][j]$.

48.2 Two-dimensional operations

A two-dimensional Fenwick tree simply nests the one-dimensional logic.

```

ADD(x, y, delta):
    i = x
    while i <= n:
        j = y
        while j <= m:
            t[i][j] += delta
            j += lowbit(j)
        i += lowbit(i)

PREFIX-SUM(x, y):
    ans = 0
    i = x
    while i > 0:
        j = y
        while j > 0:
            ans += t[i][j]
            j -= lowbit(j)
        i -= lowbit(i)
    return ans

```

A rectangle query over $[x_1, x_2] \times [y_1, y_2]$ follows by inclusion-exclusion:

$$\begin{aligned} \text{sum}(x_1, x_2, y_1, y_2) &= S(x_2, y_2) - S(x_1 - 1, y_2) \\ &\quad - S(x_2, y_1 - 1) + S(x_1 - 1, y_1 - 1). \end{aligned}$$

Theorem 48.1 (Complexity in dimension d). *A d -dimensional Fenwick tree supports point update and orthogonal prefix query in*

$$\mathcal{O}(\log^d n)$$

time for side length n in each dimension, and uses $\mathcal{O}(n^d)$ memory if stored densely.

Proof. Each coordinate contributes an independent Fenwick loop of length $\mathcal{O}(\log n)$. Nesting d such loops gives $\mathcal{O}(\log^d n)$ operations. A dense table stores one cell for every grid point, hence $\mathcal{O}(n^d)$ memory. $\square$

Remark 48.1. For sparse point sets, one often combines coordinate compression with maps or uses a Fenwick tree of Fenwick trees, discussed later.

Common pitfalls. The number of inclusion-exclusion terms for a d -dimensional orthogonal box is 2^d . The logarithmic part is $\log^d n$, but the query formula also has this constant factor.

Exercise 48.1. Write the inclusion-exclusion formula for a three-dimensional box query.

49 Reverse Fenwick tree: range maximum and point increase

49.1 Problem

The standard Fenwick tree works naturally for invertible group operations such as addition. For maximum, subtraction is impossible. Still, a monotone version works:

- point update: $a[i] \leftarrow \max(a[i], x)$, so values only increase;
- prefix maximum or suffix/range maximum depending on orientation.

A common reverse variant supports:

- `increase(i, x)`: set $a[i] \leftarrow \max(a[i], x)$;
- `suffixMax(i)`: return $\max\{a[j] : j \geq i\}$.

49.2 Reverse intervals

Store in $t[i]$ the maximum over the interval

$$[i, i + \text{lowbit}(i) - 1].$$

Then suffix queries move upward in the opposite direction.

```

INCREASE(i, x):
    while i > 0:
        t[i] = max(t[i], x)
        i -= lowbit(i)

SUFFIX-MAX(i):
    ans = -infinity
    while i <= n:
        ans = max(ans, t[i])
        i += lowbit(i)
    return ans

```

Proposition 49.1 (Correctness under monotone updates). *If updates only increase point values, the reverse Fenwick invariant*

$$t[i] = \max_{j \in [i, i + \text{lowbit}(i) - 1]} a[j]$$

remains valid after each update.

Proof. A cell $t[p]$ is affected by increasing $a[i]$ exactly if $i \in [p, p + \text{lowbit}(p) - 1]$. The reverse update loop visits precisely these containers. Since $a[i]$ only increases, replacing each visited cell by the maximum of its old value and the new value is sufficient. No visited cell can require a decrease, and no unvisited cell contains i . $\square$

Remark 49.1. This is not a fully dynamic range maximum data structure. If values can decrease, old maxima may become invalid, and Fenwick cells do not store enough information to repair them in $\mathcal{O}(\log n)$ time.

Common pitfalls. For maximum, monotonicity is essential. Point assignment or point decrease requires a segment tree or a more complicated structure.

Exercise 49.1. Write the symmetric variant that supports prefix maximum $\max\{a[1], \dots, a[i]\}$ with point increases.

50 Fenwick tree of Fenwick trees

50.1 Motivation

A dense two-dimensional Fenwick tree costs $\mathcal{O}(nm)$ memory. If we have only q relevant points, this is wasteful. A Fenwick tree of Fenwick trees stores, for each outer Fenwick index, only the inner coordinates that can actually appear there.

50.2 Offline construction

Suppose all possible point updates (x, y) are known offline. For each update point, insert y into every outer Fenwick node that would be touched by an update at x .

```
PREPARE(points):
    for each (x, y) in points:
        i = x
        while i <= n:
            coords[i].push_back(y)
            i += lowbit(i)

    for i = 1..n:
        sort and deduplicate coords[i]
        bit[i] = Fenwick tree of size |coords[i]|
```

Then a point update performs an outer Fenwick update, and inside each visited node it performs a compressed inner Fenwick update.

```
ADD(x, y, delta):
    i = x
    while i <= n:
        j = lower_bound(coords[i], y) + 1
        bit[i].ADD(j, delta)
        i += lowbit(i)

PREFIX-SUM(x, y):
    ans = 0
    i = x
    while i > 0:
        j = upper_bound(coords[i], y)
        ans += bit[i].PREFIX-SUM(j)
        i -= lowbit(i)
    return ans
```

Invariant. Outer node i stores information for all points whose x -coordinate belongs to the Fenwick interval $(i - \text{lowbit}(i), i]$. Its inner Fenwick tree is indexed by the y -coordinates that can occur in that outer node.

Theorem 50.1 (Complexity). *If there are q offline update points, the total number of stored inner coordinates is $\mathcal{O}(q \log n)$. Each point update or prefix query costs $\mathcal{O}(\log^2 n)$ time, assuming binary search inside coordinate lists.*

Proof. Each point (x, y) is inserted into $\mathcal{O}(\log n)$ outer Fenwick nodes, so the total coordinate-list size is $\mathcal{O}(q \log n)$. An operation visits $\mathcal{O}(\log n)$ outer nodes. At each one it performs a binary search and an inner Fenwick operation, each costing $\mathcal{O}(\log q)$ in the worst case. For $q = \mathcal{O}(n^2)$ this is conventionally written as $\mathcal{O}(\log^2 n)$ after normalization of coordinate sizes. $\square$

Remark 50.1. The same construction can be phrased as a two-dimensional offline Fenwick tree with coordinate compression in the second dimension separately for each first-dimensional Fenwick node.

Common pitfalls. This structure is usually offline. If new y -coordinates appear later in a node that did not reserve them, the compressed inner Fenwick tree cannot insert them without rebuilding.

Exercise 50.1. Explain how to answer rectangle-sum queries using four prefix sums in a Fenwick tree of Fenwick trees.

51 Binary search tree: definition and operations

51.1 Definition

A *binary search tree* (BST) stores keys from a totally ordered set. Each node v has a key $k(v)$ and left and right children satisfying:

$$\text{all keys in } v.\text{left} < k(v) < \text{all keys in } v.\text{right}.$$

For duplicate keys one must choose a convention, for example storing multiplicities or placing equal keys consistently on one side.

Mental model. A BST is a dynamic sorted sequence represented by recursive pivots. At every node, comparison with the key tells you which half may contain the answer.

51.2 Operations without implementation details

The basic operations are:

- $\text{find}(x)$: search for key x ;
- $\text{insert}(x)$: add key x while preserving the BST property;
- $\text{erase}(x)$: remove one occurrence of key x ;
- $\text{merge}(T_1, T_2)$: combine two trees when all keys in T_1 are less than all keys in T_2 ;
- $\text{split}(T, x)$: split into keys $< x$ and keys $\geq x$.

51.3 Order-based queries

If every node stores subtree size, a BST can support:

- $\text{kth}(k)$: find the k -th smallest key;
- $\text{rank}(x)$: count keys $< x$;
- $\text{predecessor}(x)$: greatest key $< x$;
- $\text{successor}(x)$: smallest key $> x$.

Proposition 51.1 (Correctness of search). *The standard BST search procedure finds x if it exists and correctly reports failure otherwise.*

Proof. At a node v , if $x = k(v)$ the search succeeds. If $x < k(v)$, then by the BST property every key in the right subtree is greater than $k(v)$ and hence greater than x , so x can only be in the left subtree. The case $x > k(v)$ is symmetric. Thus each comparison discards only subtrees that cannot contain x . If the algorithm reaches a null child, the relevant candidate subtree is empty, so x is absent. $\square$

Theorem 51.1 (Complexity). *All path-based BST operations cost $\mathcal{O}(h)$ time, where h is the tree height. Without balancing, h may be n .*

Common pitfalls. A BST is not automatically logarithmic. It is logarithmic only under a balancing rule, randomization assumption, or amortized guarantee.

Exercise 51.1. Describe an insertion order of keys $1, 2, \dots, n$ that makes an ordinary unbalanced BST have height $n - 1$.

52 AVL tree: definition and depth bound

52.1 Definition

An AVL tree is a binary search tree in which every node satisfies the height-balance condition

$$|\text{height}(v.\text{left}) - \text{height}(v.\text{right})| \leq 1.$$

The *balance factor* is often defined as

$$bf(v) = \text{height}(v.\text{right}) - \text{height}(v.\text{left}).$$

Then the AVL condition is $bf(v) \in \{-1, 0, 1\}$ for every node.

Mental model. AVL trees enforce local balance so strongly that a long path forces exponentially many nodes. Therefore the height must be logarithmic.

52.2 Minimum number of nodes of height h

Let $N(h)$ be the minimum number of nodes in an AVL tree of height h . Use the convention $N(0) = 1$ for a single node and $N(1) = 2$ for height one with two levels. The smallest AVL tree of height h has one child of height $h - 1$ and the other of height $h - 2$:

$$N(h) = 1 + N(h - 1) + N(h - 2).$$

Lemma 52.1. *For all $h \geq 0$, $N(h)$ grows exponentially in h . More precisely, $N(h) \geq F_{h+2}$ for Fibonacci numbers under a suitable indexing convention.*

Proof. The recurrence for $N(h)$ dominates the Fibonacci recurrence. Since $N(h) = 1 + N(h-1) + N(h-2) \geq N(h-1) + N(h-2)$ and the initial values are positive, induction gives a Fibonacci lower bound. Fibonacci numbers satisfy $F_h = \Theta(\varphi^h)$ where $\varphi = (1 + \sqrt{5})/2 > 1$, so $N(h) \geq c\varphi^h$ for some constant $c > 0$. $\square$

Theorem 52.1 (AVL height bound). *An AVL tree with n nodes has height $\mathcal{O}(\log n)$.*

Proof. If the height is h , then $n \geq N(h) \geq c\varphi^h$. Taking logarithms gives

$$h \leq \log_{\varphi}(n/c) = \mathcal{O}(\log n).$$

$\square$

Common pitfalls. The AVL condition is local but the height bound is global. The proof works because the local condition implies the Fibonacci recurrence for minimal subtree sizes.

Exercise 52.1. Compute $N(0), N(1), N(2), N(3), N(4)$ and compare with Fibonacci numbers.

53 Fixing imbalance in an AVL tree

53.1 Rotations

AVL trees restore balance by rotations. A rotation changes local parent-child relations while preserving in-order key order.

A right rotation at node z with left child y has the schematic form:

$$\begin{array}{ccc} z & \longrightarrow & y \\ \swarrow & & \swarrow \\ yC & & Az \\ \swarrow & & \swarrow \\ AB & & BC \end{array}$$

The key order is

$$A < y < B < z < C,$$

so the BST property is preserved.

53.2 Four imbalance cases

Let z be the lowest node whose balance becomes invalid.

- Left-left case: rotate right at z .
- Right-right case: rotate left at z .
- Left-right case: rotate left at $z.\text{left}$, then rotate right at z .
- Right-left case: rotate right at $z.\text{right}$, then rotate left at z .

Proposition 53.1 (Rotations preserve BST order). *Single and double AVL rotations preserve the in-order sequence of keys.*

Proof. For a single right rotation, before rotation the in-order sequence is

$$A, y, B, z, C.$$

After rotation it is still

$$A, y, B, z, C.$$

Thus the BST property is preserved. A left rotation is symmetric. A double rotation is a composition of two single rotations, so it also preserves in-order order. $\square$

Proposition 53.2 (Local balance restoration). *In each of the four AVL cases, the corresponding rotation or double rotation restores the AVL condition at the involved nodes, assuming the imbalance was caused by a single insertion or deletion step below them.*

Proof. The proof is a finite case analysis on the heights of the subtrees A, B, C and, for double rotations, the two grandchildren of the middle node. In the left-left case, the left subtree of z is too tall, and its left child is at least as high as its right child. A right rotation moves y up and z down, reducing the height of the heavy side and increasing the height of the light side at the local root. The resulting height differences are at most 1. The right-right case is symmetric. In the left-right and right-left cases, the first rotation converts the situation into an outer case; the second rotation restores balance. Because only subtrees whose heights differ by at most one are rearranged, the final local balance factors belong to $\{-1, 0, 1\}$. $\square$

Common pitfalls. After a rotation, update child pointers first and heights second. Height recomputation must occur bottom-up: first the lower node, then the new local root.

Exercise 53.1. Draw the left-right case and verify explicitly that in-order order is preserved.

54 AVL tree: find, insert, and erase

54.1 Find

find is ordinary BST search, so it costs $\mathcal{O}(h) = \mathcal{O}(\log n)$ in an AVL tree.

54.2 Insertion

Insert as in a BST, then walk back to the root, recomputing heights and rebalancing every node on the search path.

```
INSERT(v, x):
    if v == null:
        return new Node(x)

    if x < v.key:
        v.left = INSERT(v.left, x)
    else if x > v.key:
```

```

        v.right = INSERT(v.right, x)
    else:
        handle duplicate key

    UPDATE-HEIGHT(v)
    return REBALANCE(v)

```

54.3 Deletion

Deletion first performs ordinary BST deletion. If the node has two children, replace its key by the successor key and delete that successor from the right subtree. Then rebalance while returning from recursion.

```

ERASE(v, x):
    if v == null:
        return null

    if x < v.key:
        v.left = ERASE(v.left, x)
    else if x > v.key:
        v.right = ERASE(v.right, x)
    else:
        if v.left == null: return v.right
        if v.right == null: return v.left

        s = MINIMUM(v.right)
        v.key = s.key
        v.right = ERASE(v.right, s.key)

    UPDATE-HEIGHT(v)
    return REBALANCE(v)

```

54.4 Rebalance routine

```

REBALANCE(v):
    if balance(v) == -2:
        if balance(v.left) > 0:
            v.left = ROTATE-LEFT(v.left)
        return ROTATE-RIGHT(v)

    if balance(v) == +2:
        if balance(v.right) < 0:
            v.right = ROTATE-RIGHT(v.right)
        return ROTATE-LEFT(v)

    return v

```

Theorem 54.1 (Correctness). *AVL insertion and deletion preserve both the BST property and the AVL balance condition.*

Proof. The initial recursive insertion or deletion is the standard BST operation, so it preserves in-order ordering except for the intended addition or removal of one key. During the return phase, rotations preserve in-order order by the previous section, hence the BST property remains true.

Only nodes on the search path can have changed subtree heights. The algorithm visits exactly those nodes on the way back. At each such node it recomputes the height from children and applies the appropriate local rebalance if needed. By the local correctness of AVL rotations, the subtree rooted at that node becomes AVL again. Inducting upward along the recursion stack shows that the final root is an AVL tree containing exactly the desired key set. $\square$

Theorem 54.2 (Complexity). *find, insert, and erase in an AVL tree cost $\mathcal{O}(\log n)$ time. The memory overhead is $\mathcal{O}(1)$ per node for height or balance factor.*

Proof. The height is $\mathcal{O}(\log n)$. Each operation follows one root-to-leaf path and performs $\mathcal{O}(1)$ work per visited node, including at most a constant number of rotations at each level. $\square$

Common pitfalls. Deletion may require rebalancing several ancestors, not just the first imbalanced node. Insertion often stops earlier in optimized implementations, but the simple recursive version can safely rebalance all the way to the root.

Exercise 54.1. Explain why replacing a deleted node by its successor preserves the BST order.

55 Splay tree: definition and practical significance

55.1 Definition

A splay tree is a binary search tree equipped with a restructuring operation `splay(x)` that moves a node x to the root by a sequence of rotations. Unlike AVL trees, splay trees store no explicit balance information.

Definition 55.1 (Splay tree). A splay tree is a BST in which every access, insertion, or deletion performs splaying so that a recently accessed relevant node becomes the root.

55.2 Practical significance

Splay trees are important because they give $\mathcal{O}(\log n)$ amortized time per operation without storing priorities, colors, heights, or balance factors. They also have locality properties: frequently accessed elements tend to remain near the root.

Mental model. A splay tree is self-adjusting. It does not maintain an obvious local balance condition. Instead, it repairs the tree by aggressively moving accessed nodes to the root. The analysis is amortized, not worst-case per operation.

55.3 Typical applications

Splay trees are useful when:

- access patterns have locality;
- split and merge are frequent;
- one wants a balanced-BST interface with small metadata;

- amortized guarantees are acceptable.

Remark 55.1. A single splay operation may cost $\Theta(n)$ in the worst case. The guarantee is that any sequence of m operations costs $\mathcal{O}((m + n) \log n)$ under the standard model.

Common pitfalls. Do not claim that splay trees are height-balanced. They are not. Their power is amortized balance under sequences of operations.

Exercise 55.1. Give an example of a BST shape in which splaying the deepest node costs $\Theta(n)$ rotations.

56 Splay operations: zig, zig-zig, zig-zag, and splay

56.1 Primitive rotations

Let x be a node, p its parent, and g its grandparent if it exists. Splaying repeatedly applies one of three cases.

56.2 Zig

If x has a parent but no grandparent, rotate x once over its parent. This makes x the root.

56.3 Zig-zig

If x and its parent p are both left children, or both right children, perform two rotations in the same direction: first rotate p over g , then rotate x over p .

56.4 Zig-zag

If x is a left child and p is a right child, or vice versa, perform two rotations in opposite directions: first rotate x over p , then rotate x over g .

```
SPLAY(x):
    while x.parent != null:
        p = x.parent
        g = p.parent

        if g == null:
            ROTATE(x)           // zig
        else if (x is left child of p) == (p is left child of g):
            ROTATE(p)           // zig-zig, first rotation
            ROTATE(x)           // zig-zig, second rotation
        else:
            ROTATE(x)           // zig-zag, first rotation
            ROTATE(x)           // zig-zag, second rotation
```

Here ROTATE(x) means rotate x over its parent, preserving BST order.

Proposition 56.1 (Correctness of splay). *SPLAY(x) preserves the BST property and terminates with x at the root.*

Proof. Each primitive rotation preserves the in-order sequence of keys, as for AVL rotations. Therefore every step preserves the BST property.

Each iteration either performs a zig step and finishes, or performs two rotations that move x up by two levels. Thus the depth of x strictly decreases. Since the depth is a nonnegative integer, the process terminates. At termination x has no parent, hence it is the root. $\square$

Remark 56.1. The order of rotations in zig-zig is crucial for the amortized analysis. Rotating x first twice would still move x upward, but it would not give the same potential decrease.

Common pitfalls. After every rotation, parent pointers and child pointers must be updated consistently. Most implementation bugs in splay trees are pointer-maintenance bugs, not algorithmic mistakes.

Exercise 56.1. Draw the zig-zig case for two consecutive left edges and write the in-order sequence before and after the two rotations.

57 Potential method

57.1 Amortized analysis by potential

The potential method assigns a nonnegative number $\Phi(D)$ to every state D of a data structure. If operation i has actual cost c_i and changes the state from D_{i-1} to D_i , its amortized cost is

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Theorem 57.1 (Potential telescoping). *For any sequence of m operations,*

$$\sum_{i=1}^m c_i = \sum_{i=1}^m \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

In particular, if $\Phi(D_m) \geq \Phi(D_0)$ or more generally $\Phi(D) \geq 0$ and $\Phi(D_0) = 0$, then

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m \hat{c}_i.$$

Proof. By definition,

$$c_i = \hat{c}_i - \Phi(D_i) + \Phi(D_{i-1}).$$

Summing over i gives a telescoping sum:

$$\sum_{i=1}^m c_i = \sum_{i=1}^m \hat{c}_i + \sum_{i=1}^m (\Phi(D_{i-1}) - \Phi(D_i)) = \sum_{i=1}^m \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

The inequality follows from the stated nonnegativity assumptions. $\square$

57.2 How to choose a potential

A useful potential stores prepaid work in the data structure. It should increase when an operation makes the structure worse and decrease when a later expensive operation repairs it.

Example 57.1 (Dynamic array). After expansions, a dynamic array has unused capacity. A common potential is proportional to the number of stored elements minus a fraction of capacity. It stores credits for future copying.

Common pitfalls. The amortized cost of a single operation can be negative. This is not a problem. Only the sum over a sequence needs to upper-bound the total actual cost.

Exercise 57.1. Let $\Phi(D)$ be the number of elements in the input stack of a two-stack queue. Use the potential method to prove $\mathcal{O}(1)$ amortized queue operations.

58 Amortized time of splay via the potential method

58.1 Rank potential

For a node v , let

$$s(v) = \text{number of nodes in the subtree rooted at } v,$$

and define its rank

$$r(v) = \log s(v).$$

The potential of the whole tree is

$$\Phi(T) = \sum_{v \in T} r(v) = \sum_{v \in T} \log s(v).$$

The base of the logarithm is irrelevant up to constant factors.

Mental model. A node deep in an unbalanced chain has many ancestors with large subtrees. Splaying it changes subtree sizes. The rank potential measures how much hidden imbalance is stored in the tree.

58.2 Access lemma

The central theorem is the access lemma.

Theorem 58.1 (Access lemma). *Let x be splayed in a tree with root t . The amortized cost of $\text{SPLAY}(x)$ is at most*

$$3(r(t) - r(x)) + 1.$$

In particular, since $r(t) \leq \log n$ and $r(x) \geq 0$, splaying costs $\mathcal{O}(\log n)$ amortized time.

Proof sketch with the key inequality. The actual cost is counted as the number of rotations. The analysis groups rotations into zig, zig-zig, and zig-zag steps.

For a zig step, one rotation is performed. The potential change is $r'(x) + r'(p) - r(x) - r(p)$, where primes denote ranks after the rotation. Since x becomes the root of the local subtree, $r'(x) = r(p)$ before the rotation, and the step is bounded by

$$1 + \Delta\Phi \leq 1 + 3(r'(x) - r(x)).$$

For a zig-zig or zig-zag step, two rotations are performed. A standard logarithmic concavity argument gives

$$2 + \Delta\Phi \leq 3(r'(x) - r(x)).$$

The inequality follows from the fact that the two smaller side subtrees cannot both be large compared with the final subtree of x ; equivalently, it is a consequence of

$$\log a + \log b \leq 2 \log \frac{a+b}{2}$$

for positive a, b .

Summing these inequalities over all splay steps telescopes in the rank of x : the intermediate terms cancel, leaving at most

$$3(r(t) - r(x)) + 1.$$

Here $r(t)$ is the rank of the original root, equal to $\log n$ when the tree has n nodes. Thus the amortized cost is $\mathcal{O}(\log n)$. $\square$

Corollary 58.1. *Any sequence of m splay operations on an n -node tree costs $\mathcal{O}(m \log n + n \log n)$ actual time under the standard potential accounting.*

Remark 58.1. The theorem does not say every splay has logarithmic actual cost. It says the expensive splays are paid for by potential decreases accumulated from earlier structure changes.

Common pitfalls. The access lemma depends on the specific zig-zig order: rotate the parent first, then the node. This is one reason splay is not just arbitrary repeated rotation to the root.

Exercise 58.1. For a path-shaped tree of n nodes, estimate the initial potential $\sum_v \log s(v)$.

59 Splay tree: insert, erase, and find

59.1 Find

To find a key x , perform ordinary BST search. If x is found, splay that node to the root. If x is not found, splay the last visited node. The second convention is useful because it moves the predecessor or successor region near the root.

```

FIND(root, x):
    v = root
    last = null
    while v != null:
        last = v
        if x == v.key:
            SPLAY(v)
            return v
        else if x < v.key:
            v = v.left
        else:
            v = v.right

    if last != null:
        SPLAY(last)
    return null

```


59.2 Insert

One common implementation inserts as in a BST and then splays the inserted node.

```

INSERT(root, x):
    if root == null:
        return new Node(x)

    v = root
    while true:
        if x < v.key:
            if v.left == null:
                v.left = new Node(x, parent = v)
                SPLAY(v.left)
                return v.left
            v = v.left
        else if x > v.key:
            if v.right == null:
                v.right = new Node(x, parent = v)
                SPLAY(v.right)
                return v.right
            v = v.right
        else:
            SPLAY(v)
            return v

```

59.3 Erase

To erase x , first find and splay it. If found, it becomes the root. Then merge its left and right subtrees.

```

ERASE(root, x):
    v = FIND(root, x)
    if v == null or v.key != x:
        return current root

    L = v.left
    R = v.right
    if L != null: L.parent = null
    if R != null: R.parent = null
    delete v
    return MERGE(L, R)

```

Proposition 59.1 (Correctness). *The splay-tree implementations of find, insert, and erase preserve the BST property and implement the intended set operations.*

Proof. Search follows the ordinary BST comparison rule, so it finds the key exactly when it exists. Splaying preserves in-order order, so it does not change the represented set.

Insertion attaches a new key at the unique null child where BST search for that key terminates. This is exactly the standard BST insertion location, so the BST property holds before splaying; after splaying it still holds.

Deletion first splays the target key to the root. The left subtree then contains exactly keys smaller than x , and the right subtree contains exactly keys greater than x . Re-

moving the root and merging these two ordered subtrees gives the original set without x . $\square$

Theorem 59.1 (Amortized complexity). *The operations `find`, `insert`, and `erase` take $\mathcal{O}(\log n)$ amortized time.*

Proof. Each operation consists of a constant number of ordinary downward searches, rotations, splays, and merges/splits depending on implementation. The access lemma gives $\mathcal{O}(\log n)$ amortized time per splay. Other pointer operations are linear in the same searched path or constant after splaying. Therefore the total amortized time is $\mathcal{O}(\log n)$. $\square$

Common pitfalls. After `FIND` fails, the root may have changed because the last visited node is splayed. The caller must use the current root, not an old root pointer.

Exercise 59.1. Explain why after splaying key x to the root, every key in the left subtree is $< x$ and every key in the right subtree is $> x$.

60 Splay tree: merge and split

60.1 Merge

Assume all keys in A are smaller than all keys in B . To merge:

1. If one tree is empty, return the other.
2. In A , find the maximum node and splay it to the root.
3. This root has no right child.
4. Attach B as its right child.

```

MERGE(A, B):
    if A == null: return B
    if B == null: return A

    x = MAXIMUM(A)
    SPLAY(x)           // x becomes root of A
    x.right = B
    B.parent = x
    return x

```

Proposition 60.1 (Correctness of merge). *If every key in A is less than every key in B , then `MERGE(A, B)` returns a BST containing exactly the union of the two key sets.*

Proof. Splaying the maximum of A preserves the key set and makes that maximum the root. Since it is the maximum, it has no right child. All remaining keys in A are in its left subtree and are smaller than it. Every key in B is larger than every key in A , hence larger than the root. Attaching B as the right child preserves the BST property and includes exactly all keys from both trees. $\square$

60.2 Split

To split a tree T by key x into

$$A = \{k : k < x\}, \quad B = \{k : k \geq x\},$$

search for x and splay the found node or the last visited node. Then separate around the root depending on its key.

```

SPLIT(T, x):
    if T == null:
        return (null, null)

    FIND(T, x)          // splays found or last visited node
    root = current root

    if root.key < x:
        A = root
        B = root.right
        if B != null: B.parent = null
        A.right = null
        return (A, B)
    else:
        B = root
        A = root.left
        if A != null: A.parent = null
        B.left = null
        return (A, B)

```

Proposition 60.2 (Correctness of split). *SPLIT(T, x) returns two BSTs containing exactly the keys $< x$ and $\geq x$.*

Proof. After the search and splay, the root is either x itself or the last node encountered by the search. If the root key is $< x$, then by the BST search path property there is no key $\geq x$ in the root's left subtree, and any keys $\geq x$ must lie in the right subtree. Detaching the right subtree therefore separates keys $< x$ from keys $\geq x$.

If the root key is $\geq x$, the symmetric argument shows that all keys $< x$ lie in the left subtree, and the root together with its right subtree contains keys $\geq x$. Detaching the left subtree gives the desired split. Splaying and detaching preserve internal BST order in each part. $\square$

Theorem 60.1 (Amortized complexity). *merge and split in a splay tree take $\mathcal{O}(\log n)$ amortized time.*

Proof. merge performs one search for the maximum and one splay, followed by constant pointer updates. split performs one search/splay and constant pointer updates. By the access lemma, each splay costs $\mathcal{O}(\log n)$ amortized time. $\square$

Common pitfalls. The precondition for merge is essential: all keys in the first tree must be smaller than all keys in the second. Without this condition, attaching one tree as a child violates the BST property.

Exercise 60.1. Use split and merge to implement insertion of a new key x without first doing ordinary BST insertion.

Summary table

Structure	Main operations	Complexity
Dynamic segment tree	point update, range query over huge universe	$\mathcal{O}(\log U)$ time, memory proportional to touched paths
Coordinate-compressed segment tree	offline large coordinates	$\mathcal{O}(m)$ memory, $\mathcal{O}(\log m)$ operation time
Persistent segment tree	versioned point updates and queries	$\mathcal{O}(\log n)$ time and new nodes per update
Persistent range counting	count values $\leq x$ on $[l, r]$	$\mathcal{O}(\log m)$ per query after $\mathcal{O}(n \log m)$ preprocessing
Persistent range k-th statistic	k-th smallest on $[l, r]$	$\mathcal{O}(\log m)$ per query
Fenwick tree	point add, prefix sum	$\mathcal{O}(\log n)$ time, $\mathcal{O}(n)$ memory
d -dimensional Fenwick tree	point add, box prefix sum	$\mathcal{O}(\log^d n)$ time
Fenwick of Fenwicks	sparse offline 2D sums	$\mathcal{O}(\log^2 n)$ time, $\mathcal{O}(q \log n)$ memory
BST	ordered dictionary operations	$\mathcal{O}(h)$, worst-case $\mathcal{O}(n)$ if unbalanced
AVL tree	balanced ordered dictionary	$\mathcal{O}(\log n)$ worst-case
Splay tree	self-adjusting ordered dictionary	$\mathcal{O}(\log n)$ amortized

Checklist of core ideas

- A dynamic segment tree saves memory by treating missing nodes as neutral intervals.
- Coordinate compression preserves order, not distances.
- Persistent segment trees work by path copying and immutable shared subtrees.
- Prefix persistent trees turn subarray queries into differences of versions.
- Parallel descent finds order statistics by comparing left-subtree frequency differences.
- A Fenwick tree stores sums over intervals determined by the least significant set bit.
- Higher-dimensional Fenwick trees multiply logarithmic factors.
- Fenwick trees for maximum require monotone updates unless additional information is stored.
- A BST gives logarithmic operations only if height is logarithmic.
- AVL balance implies a Fibonacci lower bound on the number of nodes of a given height.

- Rotations preserve in-order order; balancing algorithms rely on this fact.
- Splay trees are not height-balanced; they are amortized-balanced via a potential argument.
- The splay access lemma is the technical core behind all splay-tree operation bounds.

61 Implicit-key search tree: sequence operations and range sums

61.1 Problem

An ordinary binary search tree stores elements ordered by explicit keys. An *implicit-key search tree* stores a sequence

$$A[0], A[1], \dots, A[n-1]$$

where the key of an element is its current position. The position is not stored explicitly; it is computed from subtree sizes.

The model operations are:

- insert an element at position i ;
- erase the element at position i ;
- assign or add to the element at position i ;
- compute the sum on an interval $[l, r)$.

The most common implementation is an *implicit treap*. The same idea can also be implemented with an implicit splay tree.

Mental model. The tree is a balanced binary tree whose in-order traversal is the array. The index of a node is determined by how many nodes lie to its left.

61.2 Augmented node data

Each node stores

$$x, \text{ priority}, \text{ left}, \text{ right}, \text{ size}, \text{ sum}.$$

The local recomputation procedure is

$$\text{size}(v) = 1 + \text{size}(v.\text{left}) + \text{size}(v.\text{right}),$$

$$\text{sum}(v) = \text{sum}(v.\text{left}) + v.x + \text{sum}(v.\text{right}).$$

Null children have size 0 and sum 0.

Invariant. For every node v , the in-order traversal of its subtree is exactly the corresponding contiguous segment of the represented sequence. Also, $\text{size}(v)$ and $\text{sum}(v)$ are correct for this subtree.

61.3 Split and merge by implicit position

The primitive operation is

$$\text{split}(T, k) = (L, R),$$

where L contains the first k elements of T , and R contains the remaining elements.

```

SPLIT(T, k):          // first k elements go to L
    if T == null:
        return (null, null)

    leftSize = SIZE(T.left)
    if k <= leftSize:
        (L, newLeft) = SPLIT(T.left, k)
        T.left = newLeft
        PULL(T)
        return (L, T)
    else:
        (newRight, R) = SPLIT(T.right, k - leftSize - 1)
        T.right = newRight
        PULL(T)
        return (T, R)

MERGE(A, B):          // every element of A precedes every element of B
    if A == null: return B
    if B == null: return A

    if A.priority < B.priority:
        A.right = MERGE(A.right, B)
        PULL(A)
        return A
    else:
        B.left = MERGE(A, B.left)
        PULL(B)
        return B

```

Here smaller priority means closer to the root. The operation MERGE assumes that all elements of A must appear before all elements of B in the sequence.

Proposition 61.1 (Correctness of implicit split). *SPLIT(T, k) returns two trees whose in-order traversals are the first k elements and the remaining elements of T .*

Proof. The proof is by induction on the size of T . Let $s = \text{size}(T.\text{left})$. If $k \leq s$, then the cut lies inside the left subtree. By induction, splitting $T.\text{left}$ into $(L, \text{newLeft})$ correctly separates the first k elements of the left subtree from the rest. The rest of the left subtree, then T , then the right subtree form the suffix, so assigning $T.\text{left} = \text{newLeft}$ gives the correct right part. The case $k > s$ is symmetric: the left subtree and the root must belong to the prefix, and the remaining cut is in the right subtree at position $k - s - 1$. After each pointer change, PULL restores the augmented values. $\square$

61.4 Sequence operations

```

INSERT(T, i, x):
    node = new Node(x)
    (A, B) = SPLIT(T, i)
    return MERGE(MERGE(A, node), B)

```

```

ERASE(T, i):
    (A, B) = SPLIT(T, i)
    (mid, C) = SPLIT(B, 1)
    delete mid
    return MERGE(A, C)

SET(T, i, x):
    (A, B) = SPLIT(T, i)
    (mid, C) = SPLIT(B, 1)
    mid.x = x
    PULL(mid)
    return MERGE(A, MERGE(mid, C))

RANGE-SUM(T, l, r):
    (A, B) = SPLIT(T, l)
    (M, C) = SPLIT(B, r - l)
    ans = SUM(M)
    T = MERGE(A, MERGE(M, C))
    return (ans, T)

```

Theorem 61.1 (Complexity). *With random priorities, all listed operations run in expected $\mathcal{O}(\log n)$ time. The extra memory per node is $\mathcal{O}(1)$.*

Proof idea. Each operation uses a constant number of splits and merges. In a treap with independent random priorities, expected height is $\mathcal{O}(\log n)$. Split and merge follow one search path, hence take time proportional to the height. $\square$

Common pitfalls. The value i is not stored as a field. If positions were stored explicitly, inserting at the front would require changing all later keys. Subtree sizes avoid this problem.

Exercise 61.1. Extend the implicit treap to support reversing a subarray $[l, r)$ using one lazy flag per node. State the new invariant and describe the PUSH procedure.

62 Hash functions, collisions, universal and k -independent families

62.1 Hashing model

A hash function maps keys from a large universe U into a table of size m :

$$h : U \rightarrow \{0, 1, \dots, m-1\}.$$

A *collision* is an event

$$x \neq y, \quad h(x) = h(y).$$

Since usually $|U| \gg m$, collisions are unavoidable by the pigeonhole principle. The algorithmic goal is not to avoid all collisions, but to make collisions sufficiently unlikely or easy to resolve.

Definition 62.1 (Universal family). A family $\mathcal{H}$ of functions $U \rightarrow \{0, \dots, m-1\}$ is *universal* if for all distinct $x, y \in U$,

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}.$$

Definition 62.2 (*k*-independent family). A family $\mathcal{H}$ is *k*-independent if for every distinct $x_1, \dots, x_k \in U$ and every $a_1, \dots, a_k \in \{0, \dots, m-1\}$,

$$\Pr[h(x_1) = a_1, \dots, h(x_k) = a_k] = m^{-k}.$$

Pairwise independence is the case $k = 2$.

Remark 62.1. Pairwise independence implies universality when the outputs are uniform, because

$$\Pr[h(x) = h(y)] = \sum_{a=0}^{m-1} \Pr[h(x) = a, h(y) = a] = m \cdot m^{-2} = 1/m.$$

62.2 Example: linear functions over a prime field

Let p be prime and $U \subseteq \mathbb{F}_p$. Choose $a, b \in \mathbb{F}_p$ uniformly with $a \neq 0$, and define

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

If $m = p$, the family $h_{a,b}(x) = ax + b$ over $\mathbb{F}_p$ is pairwise independent.

Proposition 62.1. For $m = p$, the family $h_{a,b}(x) = ax + b$ with $a, b \in \mathbb{F}_p$ chosen uniformly is pairwise independent.

Proof. Take distinct $x, y \in \mathbb{F}_p$ and desired outputs α, β . We need solve

$$ax + b = \alpha, \quad ay + b = \beta.$$

Subtracting gives

$$a(x - y) = \alpha - \beta.$$

Because $x - y \neq 0$ in a field, there is a unique a . Then $b = \alpha - ax$ is also unique. Thus among p^2 choices of (a, b) , exactly one maps (x, y) to (α, β) , so the probability is $1/p^2$. $\square$

62.3 Example: polynomial hashing

Let $\mathbb{F}$ be a finite field of size q . Choose coefficients $a_0, \dots, a_{k-1}$ uniformly and define

$$h(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1}.$$

Then h is a random polynomial of degree $< k$.

Theorem 62.1. The family of polynomials of degree $< k$ over a field is *k*-independent.

Proof. For distinct $x_1, \dots, x_k$ and arbitrary targets $y_1, \dots, y_k$, there exists a unique polynomial p of degree $< k$ satisfying $p(x_i) = y_i$ for every i by Lagrange interpolation. Since the coefficient vector is chosen uniformly from q^k possibilities, the probability of this unique polynomial is q^{-k} . $\square$

Common pitfalls. A deterministic hash function can be excellent on average for natural inputs and still be bad on adversarial inputs. Randomized hashing chooses a function from a good family after the input model is fixed.

63 Hash tables with chaining

63.1 Data structure

A hash table with chaining consists of an array of buckets

$$T[0], T[1], \dots, T[m-1],$$

where each bucket stores a linked list or dynamic array of all keys with the same hash value.

The *load factor* is

$$\alpha = \frac{n}{m},$$

where n is the current number of stored keys.

63.2 Operations

```

FIND(x):
    b = h(x)
    scan bucket T[b]
    return whether x is present

INSERT(x):
    if FIND(x) == false:
        append x to T[h(x)]
        n += 1
    if n / m > alpha_max:
        REHASH(2m)

ERASE(x):
    b = h(x)
    remove x from bucket T[b] if present
    n -= 1 if removed
    if table is too sparse:
        REHASH(m/2)

```

63.3 Expected time

Theorem 63.1. Assume h is chosen from a universal family. For any fixed key x , the expected number of other stored keys colliding with x is at most α . Therefore *FIND*, *INSERT*, and *ERASE* take expected $\mathcal{O}(1 + \alpha)$ time.

Proof. For each stored key $y \neq x$, define an indicator

$$I_y = \begin{cases} 1, & h(y) = h(x), \\ 0, & \text{otherwise.} \end{cases}$$

By universality, $\mathbb{E}[I_y] \leq 1/m$. Hence the expected number of colliding keys is

$$\mathbb{E} \left[\sum_{y \neq x} I_y \right] = \sum_{y \neq x} \mathbb{E}[I_y] \leq \frac{n-1}{m} \leq \alpha.$$

Scanning the bucket costs one plus the number of colliding keys. □

63.4 Rehashing

REHASH(newM) allocates a new table, chooses a new hash function if randomized hashing is used, and reinserts all current keys.

Theorem 63.2. *If rehashing doubles the table when α exceeds a constant and halves it when α becomes sufficiently small, then the amortized rehashing cost per update is $\mathcal{O}(1)$.*

Accounting proof. Charge each insertion a constant number of extra credits. Between a resize from m to $2m$ and the next resize, there must be $\Omega(m)$ successful updates. These credits pay for reinserting $\Theta(m)$ elements. The same argument applies to shrinking if one uses hysteresis, for example grow at $\alpha > 1$ and shrink at $\alpha < 1/4$, to avoid immediate oscillation. $\square$

Common pitfalls. If deletion is frequent, do not shrink at exactly the inverse threshold of growth. Without hysteresis, alternating insert/delete operations can trigger repeated full rehashes.

64 Perfect hashing

64.1 Static dictionary problem

Perfect hashing solves the static dictionary problem: given a fixed set S of n keys, build a data structure with no false positives and worst-case $\mathcal{O}(1)$ lookup time.

The standard construction is the Fredman-Komlos-Szemerédi two-level scheme.

64.2 Two-level construction

Choose a first-level hash function

$$h : S \rightarrow \{0, \dots, m-1\},$$

usually with $m = \Theta(n)$. Let $S_i = \{x \in S : h(x) = i\}$ and $n_i = |S_i|$.

For each bucket i , allocate a secondary table of size

$$m_i = n_i^2$$

and choose a secondary hash function h_i until it maps S_i injectively into $\{0, \dots, m_i-1\}$.

Lemma 64.1. *If h_i is chosen from a universal family and $m_i = n_i^2$, the expected number of collisions inside bucket i is less than $1/2$ for $n_i \geq 2$. Therefore a collision-free secondary function exists and is found in expected constant trials.*

Proof. The expected number of colliding unordered pairs is at most

$$\binom{n_i}{2} \frac{1}{m_i} \leq \frac{n_i^2/2}{n_i^2} = \frac{1}{2}.$$

By Markov's inequality, the probability that there is at least one collision is at most $1/2$. Hence with probability at least $1/2$ the chosen secondary hash is collision-free. $\square$

Theorem 64.1 (FKS perfect hashing). *There is a randomized construction that stores n static keys in expected $\mathcal{O}(n)$ space and supports worst-case $\mathcal{O}(1)$ lookup.*

Proof idea. The first-level function is chosen so that

$$\sum_i n_i^2 = \mathcal{O}(n).$$

For universal hashing this holds in expectation, and one can resample until it holds. Then secondary tables have total size $\sum_i n_i^2 = \mathcal{O}(n)$. Lookup computes $i = h(x)$, then checks one cell in the secondary table using $h_i(x)$. $\square$

Mental model. The first level localizes collisions. The second level spends quadratic space only inside each local bucket; because the sum of squared bucket sizes is controlled, the total remains linear.

65 Hash tables with open addressing

65.1 Data structure

Open addressing stores all keys directly in the table array. Each key x has a probe sequence

$$h(x, 0), h(x, 1), h(x, 2), \dots, h(x, m-1).$$

The operations scan this sequence until they find the key, an empty cell, or a deletion marker.

Common schemes:

- **linear probing:** $h(x, i) = (h_0(x) + i) \bmod m$;
- **quadratic probing:** $h(x, i) = h_0(x) + c_1 i + c_2 i^2 \bmod m$;
- **double hashing:** $h(x, i) = (h_1(x) + i h_2(x)) \bmod m$.

65.2 Operations with tombstones

```

FIND(x):
  for i = 0 to m-1:
    j = h(x, i)
    if T[j] == EMPTY:
      return false
    if T[j] stores x:
      return true
  return false

INSERT(x):
  firstDeleted = none
  for i = 0 to m-1:
    j = h(x, i)
    if T[j] stores x:
      return
    if T[j] == DELETED and firstDeleted == none:
      firstDeleted = j
    if T[j] == EMPTY:
      if firstDeleted != none: T[firstDeleted] = x

```

```

        else: T[j] = x
        return
    REHASH(2m)

ERASE(x):
    find the cell containing x
    if found: mark it DELETED

```

A deleted cell cannot be replaced by EMPTY, because it may lie in the middle of another key's probe sequence.

65.3 Rehashing

Open-addressed tables must control both true load and tombstone load. A typical policy is:

- grow when occupied cells exceed $\alpha_{max}m$;
- rebuild at the same size when tombstones become too numerous;
- shrink when the number of live keys is much smaller than m .

65.4 Expected-time statements

Under the idealized uniform hashing assumption, the expected number of probes is

$$\mathcal{O}\left(\frac{1}{1-\alpha}\right)$$

for unsuccessful search and insertion, and

$$\mathcal{O}\left(\frac{1}{\alpha} \log \frac{1}{1-\alpha}\right)$$

for successful search.

For double hashing, if $h_2(x)$ is relatively prime to m and the hash functions behave randomly enough, the probe sequence approximates a random permutation of table cells, so the same bounds are used as a standard model.

For linear probing, fully random hashing is not necessary, but low-independence hashing can fail. With sufficiently strong independence assumptions, linear probing has expected constant time when α is bounded away from 1. Its practical advantage is cache locality.

Common pitfalls. Linear probing is fast in practice but suffers from primary clustering: long consecutive occupied runs become more likely, and searches through such runs become expensive.

66 Bloom filter

66.1 Problem

A Bloom filter is a space-efficient probabilistic set representation. It supports

- `ADD(x)`;
- `MAYBE-CONTAINS(x)`.

It has no false negatives, but it may have false positives.

66.2 Algorithm

Use a bit array $B[0..m-1]$ and k hash functions $h_1, \dots, h_k$.

```

ADD(x):
  for j = 1 to k:
    B[h_j(x)] = 1

MAYBE-CONTAINS(x):
  for j = 1 to k:
    if B[h_j(x)] == 0:
      return false
  return true

```

Proposition 66.1 (No false negatives). *If x was inserted and bits are never reset to zero, then `MAYBE-CONTAINS(x)` returns true.*

Proof. When x is inserted, all positions $h_1(x), \dots, h_k(x)$ are set to 1. Later insertions only set additional bits to 1 and never clear existing bits. Therefore all k checked positions remain 1. $\square$

66.3 False-positive probability

After inserting n keys, the probability that a fixed bit is still zero is approximately

$$\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}.$$

Thus the probability that all k checked bits are one is approximately

$$p(k) = \left(1 - e^{-kn/m}\right)^k.$$

The optimal number of hash functions is

$$k^* = \frac{m}{n} \ln 2.$$

At this value, the false-positive probability is approximately

$$p \approx (0.6185)^{m/n}.$$

Remark 66.1. In practice one often uses double hashing to simulate k hash functions:

$$h_j(x) = h_1(x) + jh_2(x) \pmod{m}.$$

Common pitfalls. A standard Bloom filter does not support deletion. Clearing bits may create false negatives for other keys sharing those bits. Counting Bloom filters replace bits by small counters to support deletion approximately.

67 Cuckoo filter

67.1 Idea

A cuckoo filter is an approximate membership data structure similar in purpose to a Bloom filter, but it stores short fingerprints in cuckoo-hash buckets. It supports membership queries and usually deletion.

For a key x , compute:

$$f = \text{fingerprint}(x),$$

$$i_1 = h(x), \quad i_2 = i_1 \oplus h(f).$$

The fingerprint f may be stored in either bucket i_1 or bucket i_2 .

67.2 Operations

```

CONTAINS(x):
    f = fingerprint(x)
    i1 = h(x)
    i2 = i1 xor h(f)
    return f appears in bucket i1 or bucket i2

INSERT(x):
    compute f, i1, i2
    if bucket i1 or i2 has free slot:
        place f there and stop
    choose one of i1, i2
    repeat up to MaxKicks:
        evict a random fingerprint g from current bucket
        store current fingerprint there
        current fingerprint = g
        current bucket = alternate bucket of g
        if current bucket has free slot:
            place g there and stop
    rehash or rebuild

ERASE(x):
    remove fingerprint f from one of its two candidate buckets if present

```

67.3 Properties

A cuckoo filter may have false positives because two keys can have the same fingerprint. It does not have false negatives if insertions succeed and deletion is applied only to keys that are actually stored.

Mental model. A Bloom filter asks whether all required bits are set. A cuckoo filter asks whether a short fingerprint is present in one of two possible small buckets.

Remark 67.1. The precise failure probability and false-positive rate depend on bucket size, fingerprint length, load factor, and implementation details. The standard high-level guarantee is expected constant-time operations under suitable random hashing and load below a threshold.

68 Skip List

68.1 Definition

A Skip List is a randomized ordered dictionary. Each key appears in level 0 and independently rises to higher levels with probability p , commonly $p = 1/2$. Higher levels contain fewer keys and act as express lanes.

Mental model. A Skip List is a randomized analogue of a balanced binary search tree, but balancing is replaced by random tower heights.

68.2 Search

```
FIND(x):
    v = head
    for level from highest down to 0:
        while v.next[level] != null and v.next[level].key < x:
            v = v.next[level]
    v = v.next[0]
    return v != null and v.key == x
```

68.3 Insert and erase

During search, store the last node before x on every level in an array `update[level]`. For insertion, generate a random height H and splice the new node after `update[level]` for all levels $0 \leq \text{level} \leq H$. For deletion, splice the node out from every level where it appears.

```
INSERT(x):
    compute update[] by search path
    if x already exists: return
    H = random geometric height
    create node u with height H
    for level = 0 to H:
        u.next[level] = update[level].next[level]
        update[level].next[level] = u

ERASE(x):
    compute update[] by search path
    u = update[0].next[0]
    if u == null or u.key != x: return
    for level = 0 to height(u):
        if update[level].next[level] == u:
            update[level].next[level] = u.next[level]
```

Theorem 68.1 (Standard bounds). *A Skip List with geometric heights has expected space $\mathcal{O}(n)$ and expected time $\mathcal{O}(\log n)$ for find, insert, and erase.*

Proof idea. The expected number of nodes at level i is np^i , so the total expected number of pointers is $n(1+p+p^2+\dots) = \mathcal{O}(n)$. The expected search path moves right only a constant expected number of times per level, and the maximum useful level is $\mathcal{O}(\log n)$ with high probability. $\square$

69 Cartesian tree and linear construction for sorted keys

69.1 Definition

A Cartesian tree stores pairs $(key, priority)$ and satisfies two properties:

- binary-search-tree property by key;
- heap property by priority.

If priorities are all distinct, the Cartesian tree is unique.

A randomized Cartesian tree is usually called a *treap*: keys are fixed, priorities are random.

Theorem 69.1 (Expected depth). *If priorities are independent random permutations, the expected depth of each node in a treap on n keys is $\mathcal{O}(\log n)$, and the expected height is $\mathcal{O}(\log n)$ up to standard high-probability qualifications.*

Proof idea. For two keys x_i, x_j , node x_j is an ancestor of x_i exactly when x_j has the best priority among all keys in the interval between x_i and x_j . For a distance d , this probability is $1/(d+1)$. Summing over all possible ancestors gives a harmonic sum $\mathcal{O}(\log n)$ for expected depth. $\square$

69.2 Linear construction when keys are sorted

Assume keys are already sorted increasingly, so the in-order order is fixed. We only need to build the tree satisfying heap order by priorities. This can be done with a monotone stack.

```
BUILD-CARTESIAN(sorted nodes v[1..n]):
    stack = empty
    for i = 1 to n:
        last = null
        while stack not empty and stack.top.priority > v[i].priority:
            last = stack.pop()
        v[i].left = last
        if stack not empty:
            stack.top.right = v[i]
        stack.push(v[i])
    return bottom element of stack
```

Here smaller priority is higher in the heap.

Proposition 69.1. *The stack construction builds the unique Cartesian tree in $\mathcal{O}(n)$ time.*

Proof idea. Each node is pushed once and popped at most once, so the time is linear. The stack maintains the right spine of the already constructed Cartesian tree, with priorities increasing from root to bottom. When a new key arrives, it must be placed on the right boundary because its key is larger than all previous keys. Popping removes exactly those nodes whose priorities are worse than the new node's priority; they become the left subtree of the new node. The remaining stack top, if any, becomes the parent whose right child is the new node. This preserves both in-order key order and heap order. $\square$

70 Merge and split in a Cartesian tree

70.1 Merge

For treaps, $\text{MERGE}(A, B)$ assumes all keys in A are smaller than all keys in B .

```

MERGE(A, B):
  if A == null: return B
  if B == null: return A
  if A.priority < B.priority:
    A.right = MERGE(A.right, B)
    PULL(A)
    return A
  else:
    B.left = MERGE(A, B.left)
    PULL(B)
    return B

```

Proposition 70.1 (Correctness of merge). *$\text{MERGE}(A, B)$ returns a treap containing exactly all keys from A and B .*

Proof. If one tree is empty, the claim is trivial. Otherwise the root of the merged treap must be the root with better priority among the two roots, because every key in A is smaller than every key in B , and heap order requires the global best priority at the root. If A has the better root, its left subtree stays unchanged and its right subtree must become the merge of $A.\text{right}$ and B . The recursive call is correct by induction. The other case is symmetric. $\square$

70.2 Split by explicit key

```

SPLIT(T, x):          // returns keys < x and keys >= x
  if T == null:
    return (null, null)
  if T.key < x:
    (L2, R) = SPLIT(T.right, x)
    T.right = L2
    PULL(T)
    return (T, R)
  else:
    (L, R2) = SPLIT(T.left, x)
    T.left = R2
    PULL(T)
    return (L, T)

```

Proposition 70.2 (Correctness of split). *$\text{SPLIT}(T, x)$ returns two treaps containing exactly keys $< x$ and keys $\geq x$.*

Proof. If $T.\text{key} < x$, then the root and its entire left subtree belong to the left result. The only undecided keys lie in $T.\text{right}$, so the algorithm recursively splits $T.\text{right}$. The part $L2$ of that split is attached as the new right subtree of T , while R becomes the right result. Heap order is preserved because only subtrees already below T are rearranged below T . The case $T.\text{key} \geq x$ is symmetric. $\square$

Theorem 70.1 (Time). *Both operations run in expected $\mathcal{O}(\log n)$ time for randomized priorities.*

71 Insert and erase in a Cartesian tree via merge and split

71.1 Insert

To insert a new key x , split around x and merge with the new singleton node.

```
INSERT(T, x):
    if FIND(T, x): return T
    node = new Node(x, randomPriority())
    (A, B) = SPLIT(T, x)
    return MERGE(MERGE(A, node), B)
```

If duplicates are allowed, one must define a convention, for example split by (key, id) .

71.2 Erase

To erase x , isolate it by two splits.

```
ERASE(T, x):
    (A, B) = SPLIT(T, x)
    (M, C) = SPLIT(B, x + epsilon) // or split by <= x convention
    delete M
    return MERGE(A, C)
```

For integer keys one may use $SPLIT(B, x+1)$ if the first split is by $< x$. In general, it is cleaner to define two split variants: keys $< x$ and keys $\leq x$.

Proposition 71.1. *The insert and erase implementations above preserve the treap invariant.*

Proof. Split preserves the treap invariant in each result and separates key ranges. Merge preserves the treap invariant under the precondition that all keys in the left argument precede all keys in the right argument. Insert and erase are compositions of these correct operations, so they preserve the invariant. $\square$

Common pitfalls. The expression $x + \varepsilon$ is pseudocode, not an implementation for arbitrary comparable keys. Implement a precise split convention.

72 B-tree: definition, practical significance, and find

72.1 Definition

A B-tree of minimum degree $t \geq 2$ is a multiway balanced search tree satisfying:

- each node stores several sorted keys;
- every internal node with k keys has $k + 1$ children;
- all leaves have the same depth;
- except for the root, every node has at least $t - 1$ keys;

- every node has at most $2t - 1$ keys;
- the root has at least one key unless the tree is empty.

If a node has keys

$$a_1 < a_2 < \dots < a_k,$$

then its children contain keys in intervals

$$(-\infty, a_1), (a_1, a_2), \dots, (a_k, \infty).$$

72.2 Practical significance

B-trees are designed for memory hierarchies. A node is chosen to fit a disk page or cache block. Searching inside one node costs little compared to loading the node, so high branching factor greatly reduces height.

72.3 Find

```

B-TREE-FIND(v, x):
    i = lower_bound(v.keys, x)
    if i < v.numKeys and v.keys[i] == x:
        return true
    if v is leaf:
        return false
    return B-TREE-FIND(v.child[i], x)

```

Proposition 72.1 (Correctness of find). *B-TREE-FIND returns true exactly when x is stored in the B-tree.*

Proof. At a node, sorted keys divide the key universe into disjoint intervals corresponding to children. If x is equal to a key in the node, the search succeeds. Otherwise, by the B-tree search invariant, x can occur only in the unique child interval selected by `lower_bound`. Recursing into that child is therefore necessary and sufficient. If a leaf is reached and x is not present, there is no child where x could be stored. $\square$

73 Depth of a B-tree with n keys

Theorem 73.1 (Height bound). *Let T be a B-tree of minimum degree $t \geq 2$ with height h , where height means the number of edges on a root-to-leaf path. If T stores $n \geq 1$ keys, then*

$$h \leq \log_t \frac{n+1}{2}.$$

Equivalently, $h = \mathcal{O}(\log_t n)$.

Proof. The minimum number of keys in a B-tree of height h occurs when the root has one key, every other internal node has $t - 1$ keys, and every non-root internal node has t children.

At depth 0 there is one root. At depth 1 there are at least 2 nodes. At depth $d \geq 1$ there are at least $2t^{d-1}$ nodes. Each non-root node contains at least $t - 1$ keys. Therefore

$$n \geq 1 + (t - 1) \sum_{d=1}^h 2t^{d-1} = 1 + 2(t - 1) \frac{t^h - 1}{t - 1} = 2t^h - 1.$$

Thus $t^h \leq (n + 1)/2$, so

$$h \leq \log_t \frac{n + 1}{2}.$$

□

Remark 73.1. When t is large, height is very small. This is why B-trees are standard for databases and file systems.

74 B-tree insertion

74.1 Core idea

Insertion descends to the appropriate leaf. The difficulty is that a node may already contain $2t - 1$ keys. The standard algorithm avoids descending into a full child by splitting full children on the way down.

74.2 Split child

Suppose child y of node x is full, so it has $2t - 1$ keys. Its median key moves up into x , and y is split into two nodes with $t - 1$ keys each.

```
SPLIT-CHILD(x, i):
    y = x.child[i]
    z = new node

    z.keys = y.keys[t .. 2t-2]
    median = y.keys[t-1]
    y.keys = y.keys[0 .. t-2]

    if y is not leaf:
        z.children = y.children[t .. 2t-1]
        y.children = y.children[0 .. t-1]

    insert z into x.children after y
    insert median into x.keys at position i
```

74.3 Insertion algorithm

```
B-TREE-INSERT(T, x):
    r = T.root
    if r is full:
        s = new root with no keys
        s.children[0] = r
        T.root = s
        SPLIT-CHILD(s, 0)
    INSERT-NONFULL(T.root, x)
```

```

INSERT-NONFULL(v, x):
  if v is leaf:
    insert x into v.keys in sorted order
  else:
    i = child index where x belongs
    if v.child[i] is full:
      SPLIT-CHILD(v, i)
      recompute i: x may belong to the new right child
    INSERT-NONFULL(v.child[i], x)

```

Proposition 74.1 (Correctness). *B-tree insertion preserves all B-tree invariants and inserts the new key in the correct sorted position.*

Proof. Splitting a full child preserves sorted order: all keys in the left part are below the median and all keys in the right part are above it. The median inserted into the parent separates the two new children correctly. Both children receive exactly $t - 1$ keys, so they satisfy the minimum and maximum key constraints.

Before descending, the algorithm ensures the child is not full. Therefore the recursive insertion can add a key without exceeding $2t - 1$ keys, except when it further splits lower nodes before descending. Eventually insertion happens in a leaf at the unique position determined by the search intervals. All leaves remain at the same depth; the only height increase occurs when the old root is split and a new root is created above it. $\square$

75 B-tree deletion

75.1 Core idea

Deletion is harder than insertion because removing a key can cause underflow. The standard top-down algorithm maintains the invariant that before recursing into a child, that child has at least t keys. Then deleting one key inside it cannot make it fall below $t - 1$ unless the algorithm fixes the situation on the way down.

75.2 Cases

To delete key x from node v :

1. If x is in a leaf, remove it.
2. If x is in an internal node:
 - if the left child has at least t keys, replace x by its predecessor and delete the predecessor from the left child;
 - else if the right child has at least t keys, replace x by its successor and delete the successor from the right child;
 - else merge the left child, x , and the right child into one node with $2t - 1$ keys, then delete x recursively from the merged node.
3. If x is not in v , descend into the appropriate child. Before descent, if that child has only $t - 1$ keys, fix it by borrowing from a sibling with at least t keys, or by merging with a sibling.

75.3 Borrow and merge

Borrowing rotates one key through the parent:

- a separator key from the parent moves down into the deficient child;
- a boundary key from a rich sibling moves up into the parent;
- one child pointer is moved if the nodes are internal.

Merging combines two children with $t - 1$ keys and the separator key from the parent into one node with

$$(t - 1) + 1 + (t - 1) = 2t - 1$$

keys.

Proposition 75.1 (Correctness). *The B-tree deletion algorithm preserves the B-tree invariants and removes exactly the requested key if it is present.*

Proof sketch. The search interval invariant identifies the unique subtree where x can lie. If x is in a leaf, direct removal is correct. If x is in an internal node, replacing it by its predecessor or successor preserves sorted order, and the actual deletion is moved to a child where the replacement key exists. If both adjacent children have only $t - 1$ keys, merging them with the separator creates a legal full node and reduces the problem to that node.

When descending into a child not containing x at the current node, the algorithm first ensures the child has at least t keys. Borrowing preserves all key intervals and restores the key count. Merging also preserves key intervals and produces a node with $2t - 1$ keys. Thus the recursive call is always made into a sufficiently large node, and after one deletion it still has at least $t - 1$ keys. If the root becomes empty, its only child becomes the new root, decreasing the height by one while preserving equal leaf depth. $\square$

76 Red-black tree: definition

76.1 Definition

A red-black tree is a binary search tree whose nodes are colored red or black and satisfy:

1. every node is red or black;
2. the root is black;
3. every null leaf sentinel is black;
4. if a node is red, then both its children are black;
5. for every node, all paths from that node to descendant null leaves contain the same number of black nodes.

The last number is called the *black height*.

Definition 76.1 (Black height). The black height $\text{bh}(v)$ is the number of black nodes on any path from v to a descendant null leaf, usually excluding v and including the null leaf, depending on convention. Any fixed convention gives the same asymptotic results.

Mental model. A red-black tree is a BST that allows limited imbalance. Red nodes are a bookkeeping device: they permit local violations of perfect black-level balance while preventing long chains of imbalance.

77 Depth of a red-black tree

Lemma 77.1. *A subtree rooted at a node with black height b contains at least $2^b - 1$ internal nodes.*

Proof. Induct on b . If $b = 0$, the subtree is empty under the chosen convention, so the claim is true. For $b > 0$, each child subtree has black height at least $b - 1$. By induction, each child contains at least $2^{b-1} - 1$ internal nodes. Including the root gives at least

$$1 + 2(2^{b-1} - 1) = 2^b - 1.$$

□

Lemma 77.2. *If a red-black tree has height h , then its root black height is at least $h/2$.*

Proof. No red node has a red child, so along any root-to-leaf path at least half of the internal nodes are black, up to an additive constant depending on convention. Thus black height is at least $h/2$ in asymptotic and standard exact forms. □

Theorem 77.1 (Height bound). *A red-black tree with n internal nodes has height*

$$h \leq 2\log_2(n + 1).$$

Proof. Let b be the black height of the root. By the first lemma,

$$n \geq 2^b - 1.$$

Thus $b \leq \log_2(n + 1)$. By the second lemma, $h \leq 2b$, hence

$$h \leq 2\log_2(n + 1).$$

□

78 Red-black tree insertion

78.1 BST insertion plus fix-up

Insertion begins as ordinary BST insertion. The new node is colored red. This preserves black heights, but may violate the rule that a red node has black children.

```
RB-INSERT(x):
    insert x as in an ordinary BST
    color x red
    RB-INSERT-FIXUP(x)
```

78.2 Fix-up cases

Let z be the current red node whose parent p is red. Let g be its grandparent and u its uncle.

1. **Red uncle.** If u is red, recolor p and u black, recolor g red, and continue from g .
2. **Black uncle, inner case.** If z is an inner child, rotate at p to convert to the outer case.
3. **Black uncle, outer case.** Rotate at g , recolor the new subtree root black and the old grandparent red.

At the end, color the root black.

Proposition 78.1 (Correctness of insertion fix-up). *The red-black insertion fix-up restores all red-black properties.*

Proof sketch. The only possible violation after BST insertion is a red parent with a red child. In the red-uncle case, recoloring preserves the black height of every path through g : paths through p or u lose one black at the child but gain one when g becomes red/black according to the local transformation. The violation is pushed upward to g .

In the black-uncle cases, rotations preserve BST order. The final rotation makes the parent the local root and recoloring makes it black while the old grandparent becomes red. All paths through the local subtree keep the same number of black nodes, and the red-red violation is removed locally. Since the loop either moves upward or terminates, the algorithm finishes. Coloring the root black restores the root property. $\square$

79 Red-black tree deletion

79.1 BST deletion plus fix-up

Deletion first performs ordinary BST deletion. If the removed physical node was red, no black-height property changes. If a black node is removed, one subtree has a *double-black* deficit that must be fixed.

79.2 Fix-up cases

Let x be the node carrying the black deficit and let w be its sibling.

Standard cases:

1. **Red sibling.** Recolor sibling black, parent red, rotate at parent. This converts to a black-sibling case.
2. **Black sibling with two black children.** Recolor sibling red and move the deficit to the parent.
3. **Black sibling with near red child and far black child.** Rotate at sibling to convert to the far-red case.
4. **Black sibling with far red child.** Rotate at parent and recolor to eliminate the deficit.

Finally color x black.

Proposition 79.1 (Correctness of deletion fix-up). *The red-black deletion fix-up restores all red-black properties.*

Proof sketch. The invariant is that all red-black properties hold except that node x represents one missing black on paths through it. A red sibling cannot solve the deficit directly, but rotating and recoloring transforms the configuration into one with a black sibling while preserving black heights. If the sibling and its children are black, recoloring the sibling red transfers the missing black level upward. If the sibling has a red near child but black far child, a rotation at the sibling changes it into the final case. In the far-red case, a rotation at the parent and appropriate recoloring distribute black nodes so that every path has the correct black count and no red-red violation remains. The process either moves the deficit upward or removes it, so it terminates. $\square$

Common pitfalls. Deletion is much easier to implement with explicit black sentinel leaves. Null-pointer special cases otherwise obscure the symmetry of the fix-up cases.

80 Comparative analysis of search-tree implementations

80.1 Comparison table

Structure	Guarantee and operations	When it is useful
Naive BST	Operations cost $\mathcal{O}(h)$; in the worst case $h = n$. Supports find, insert, and erase.	Simple code; acceptable only when input order is benign or externally randomized.
AVL tree	Height $\mathcal{O}(\log n)$ with strict balance. Find, insert, and erase are worst-case $\mathcal{O}(\log n)$.	Fast lookup-heavy workloads; stricter balance than red-black trees, but often more rotations on updates.
Splay tree	Find, insert, erase, merge, and split are amortized $\mathcal{O}(\log n)$. A single operation may be linear.	Adaptive workloads with locality; no explicit balance field.
B-tree	Height $\mathcal{O}(\log_t n)$ for minimum degree t . Operations are block-oriented.	Databases, filesystems, external memory, and cache-aware storage.
Treap / Cartesian tree	Expected $\mathcal{O}(\log n)$ height with random priorities. Split and merge are especially simple.	Competitive programming, randomized ordered sets, and implicit-key sequences.
Red-black tree	Height at most $2\log_2(n+1)$. Find, insert, and erase are worst-case $\mathcal{O}(\log n)$.	General-purpose ordered maps and sets; weaker balance than AVL but robust update performance.

80.2 Conceptual comparison

- **Naive BST:** invariant is only order. No balance mechanism.
- **AVL:** stores height or balance factor; keeps height very small.
- **Splay:** changes the tree after every access; good recently accessed keys become cheap.

- **B-tree:** reduces height by using high branching factor and many keys per node.
- **Treap:** balance comes from random priorities; split and merge are especially clean.
- **Red-black tree:** approximates a 2-3-4 tree using colors; robust worst-case guarantees.

80.3 Which tree should one choose?

For competitive programming, treaps are often the simplest when split/merge or implicit keys are needed. For ordered dictionaries with worst-case guarantees, AVL and red-black trees are standard. For storage systems, B-trees dominate because asymptotic comparisons between keys are less important than memory transfers. For adaptive workloads, splay trees can be very effective.

Exercise 80.1. Give a sequence of insertions that makes a naive BST have height $n - 1$.

Exercise 80.2. Explain why an AVL tree is usually better for lookup-heavy workloads, while a red-black tree is often preferred for mixed update workloads.

Exercise 80.3. For each structure in the table, identify the invariant responsible for logarithmic height or amortized logarithmic time.